ADVANCED BIG DATA ANALYTICS PROJECT 2

Project: Mobile Money Investigations with PostgreSQL and Neo4j

Advanced Big Data Analytics
AUCA
Group Members:

S/N	NAMES	STUDENT ID
1	**VICTOR DUSENGIMANA**	**101372**
2	**UMWIZA ODILE**	**101365**
3	**IGIRUKWAYO ILDEPHONSE**	**101404**
4	**MUKIZA ALAIN CHRISTIAN**	**101370**

Contents

Objective 5

Methodology 5

1. Capture the data 6

1.1 Preparation of workplace 6

Software and environment specifications 6

PostgreSQL memory settings 7

Database connections 8

1.2 Build capture.py 9

Inspect the generator and preview the streams 9

Source streams and expected record counts 10

Progressive CSV capture and audit 10

Verified capture results 11

Transfer audit results 11

2. Build PostgreSQL 12

2.1 Definition of the five tables. (Pk= Primary Key) 13

1. Relational database design 14

Table 1:customers 14

Table 2: devices 14

Table 3: accounts 15

Table 4: account_devices 15

Table 5:transfers 16

Schema relationships and constraints 17

Constraint validation 18

2.2 Bulking load and checking 18

Bulk-loading the five CSV files 19

Loaded row counts 19

Index creation and analysis 19

Measured index setup times 20

Verify row counts, account roles, and transfer totals 20

Constraint and index verification 21

Database and index storage sizes 22

3. Build Neo4j 23
3.1 Mapping the same facts 23
Graph data model 23
2. Mapping source fields to the graph 25
Preserving repeated payments 25
3.2 Preparation and importing CSVs 26
Create prepare_import.py 27
Temporary file sizes 27
2. Stop the empty database and perform the full import 28
3. Start Neo4j and configure constraints and indexes 29
Neo4j node and relationship counts 29
Account role verification 30
Ownership verification 30
Transfer audit verification 31
Neo4j storage measurements 32
3.3 Inspecting one account in both models 34
2. Trace the same account using Cypher MATCH 35
3. Explain foreign keys and graph relationships 35
Why repeated payments must remain separate 36
4. Answer routine business questions 37
4.1 Q1 Account activity 37
4.2 Q2 Daily activity 38
4.3 Q3 Shared devices 39
5. Investigation of connected accounts 39
5.1 Q4 Ownership and device network 39
5.2 Q5 Rapid onward payments 41
5.3 Q6 Circular payments 42
Cycle Validation Process 42
Performance Results 42
5.4 Q7 Shortest transfer connection 43
Performance Results 43
6. Check correctness and performance 45

6.1 Building one comparison program 45
6.2 Time a fixed selection 46
6.3 Trying one SQL improvement 46
Simplified Q7 Query Execution Flow 46
7. Test concurrent requests 47
7.1 Building a small load test 47
Q4 Throughput by Client Count 48
Q4 Concurrency Results 48
Q4 p95 Latency by Client Count 48
7.2 Explanation of scaling 49
8. Making and defend decisions 50
8.1 Prioritizing investigation 50
Investigation Workflow 50
Prioritization by Recorded Depth-10 Network Size 50
8.2.0 Recommending a model 52
8.2.1 Database Architecture Recommendation 52
Recommended Architecture 53
8.3 Individual Reflections and Contributions 53
Victor Dusengimana; with Student ID 101372 53
Umwiza Odile; with Student ID 101365 54
Igirukwayo Ildephonse; with Student ID 101404 54
Mukiza Alain Christian; with Student ID 101370 54
Overall conclusion 54
APPENDIX: 55

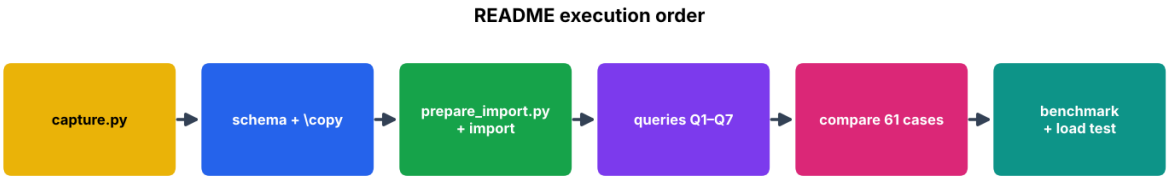
Objective

The objective is to act as analysts for a Rwandan mobile money provider: build the same payment reports and account investigations in PostgreSQL and Neo4j from the September 2026 records, compare how each model expresses the problem and how each database performs, and recommend which model suits each task, with evidence for investigation priorities.

Methodology

- Capture: a streaming capture.py writes the five generator streams to CSV without holding them in memory, and records row counts and a transfer audit (count, time range, total RWF).
- PostgreSQL: five constrained tables, bulk-loaded with \copy, then indexed and analysed; all counts and audit totals are checked against capture.
- Neo4j: typed-header import copies are produced by prepare_import.py and loaded with neo4j-admin full import; counts, roles, ownership and audit totals are checked against capture and PostgreSQL.
- Investigation: one parameterised SQL/Cypher pair per question Q1–Q7 for the six review sets A–F; answers are compared case by case (61 cases) before any timing.
- Performance: 21 timed cases per engine (3 warm-ups, 5 measured runs, perf_counter), then a concurrency test with 1, 4, 8 and 16 clients.
- Conventions: time zone Africa/Kigali (CAT); 1 GB = 1,000,000,000 bytes; no fraud is declared from these records.

1. Capture the data



1.1 Preparation of workplace

The project was developed in a Windows environment using PostgreSQL for relational data storage and Neo4j Community Edition for graph storage. Python was used to capture the generated data, prepare the import files, and support data validation.

The project directory was:

D:\Mobile_money_project_2

The original generated CSV files were stored in:

D:\Mobile_money_project_2\output

The temporary CSV files prepared for Neo4j were stored separately in:

D:\Mobile_money_project_2\neo4j_import

This separation preserved the original source files while allowing the graph database to use its required import format.

Software and environment specifications

Component	Installed configuration
Operating system	Windows 11 Pro, 64-bit
Windows build	26200
Processor	Intel Core i5-8365U
CPU cores	4 physical cores, 8 logical processors
Installed RAM	7.81 GB
Python	3.13.9
PostgreSQL	16.15
PostgreSQL connection	127.0.0.1:5433

Neo4j edition	Community Edition
Neo4j version/image	neo4j:2026.09.0
Neo4j container	mobile-money-neo4j
Neo4j database	neo4j
Neo4j Browser	http://localhost:7474
Neo4j Bolt endpoint	bolt://localhost:7687
Neo4j Python driver	6.4.1, as recorded in the project environment
PostgreSQL database	mobile_money_investigations

The Neo4j instance was run in Docker. Its database files were stored in the persistent Docker volume mobile_money_neo4j_data, mounted at /data in the container. The persistent volume allowed the imported graph database to remain available when the container was restarted.

PostgreSQL used port 5433 because the default port 5432 was occupied by another PostgreSQL instance on the computer.

PostgreSQL memory settings

The recorded PostgreSQL settings were:

Setting	Value
shared_buffers	128 MB
work_mem	4 MB
max_connections	100
Time zone required for the project	Africa/Kigali

The time zone was set for the project session using:

```
SET TIME ZONE 'Africa/Kigali';

SHOW TIME ZONE;
```

The expected result was Africa/Kigali. The Unix timestamps in the dataset represent absolute times; the time zone determines how they are displayed as local date and time values.

The following SQL commands were used to inspect PostgreSQL configuration:

```
SELECT version();

SHOW shared_buffers;
SHOW work_mem;
SHOW max_connections;
SHOW TimeZone;
```

Neo4j memory settings should be reported using the actual configuration from the running instance. No independently verified Neo4j heap and page-cache configuration values were recorded in the results available for this report; therefore, no specific values are assumed.

Database connections

PostgreSQL was accessed with:

psql -U postgres -p 5433 -d mobile_money_investigations

Neo4j was accessed through Neo4j Browser at:

http://localhost:7474

The default Neo4j database, neo4j, was used as required by the assignment.

Successful connections were verified during project execution.

Screenshots to insert:

- **Figure 1:** PostgreSQL successful connection and version.
- **Figure 2:** PostgreSQL memory settings and time zone.
- **Figure 3:** Neo4j Browser successful connection and database.
- **Figure 4:** Windows processor and RAM specifications.
- **Figure 5:** Neo4j version and relevant memory configuration.

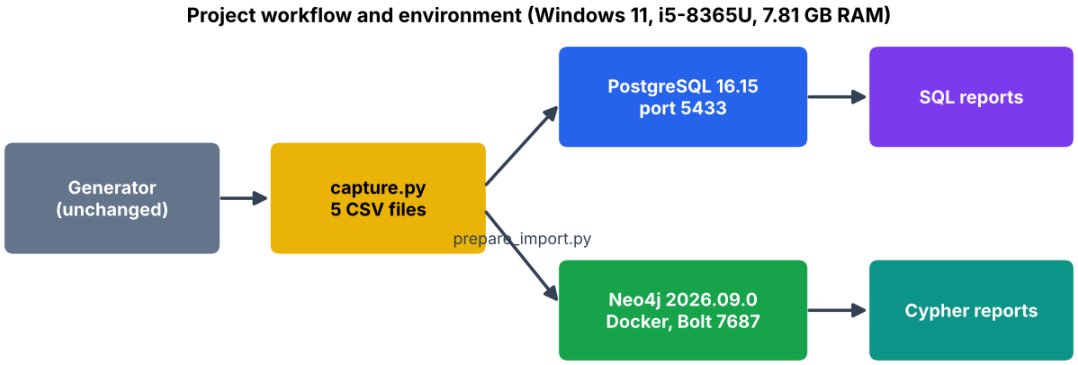


Figure 1. Project workflow and environment.

Component	Configuration
Operating system	Windows 11 Pro 64-bit (build 26200)
CPU / RAM	Intel Core i5-8365U, 4 cores / 8 logical processors; 7.81 GB RAM

Python	3.13.9; Neo4j Python driver 6.4.1
PostgreSQL	16.15, 127.0.0.1:5433 (5432 was occupied), database mobile_money_investigations
PostgreSQL settings	shared_buffers 128 MB, work_mem 4 MB, max_connections 100, time zone Africa/Kigali
Neo4j	Community Edition, image neo4j:2026.09.0, container mobile-money-neo4j, default database neo4j, Browser :7474, Bolt :7687, persistent volume mobile_money_neo4j_data

Project folder D:\Mobile_money_project_2: source CSVs in output\, temporary Neo4j copies in neo4j_import\, so the source files stay unchanged. The time zone is set with SET TIME ZONE 'Africa/Kigali' and checked with SHOW TimeZone.

1.2 Build capture.py

Inspect the generator and preview the streams

The provided generator, mobile_money_generator.py, was used without modifying its original code. It supplied the dataset metadata, field definitions, and five iterator functions.

The generator was imported using:

```
import mobile_money_generator as g
```

The functions g.dataset_info() and g.FIELDS were used to inspect the dataset and its schema. The itertools.islice function was used to preview ten records from each stream without collecting the entire dataset into memory.

Example preview code:

```
import itertools
import mobile_money_generator as g

print(g.dataset_info())
print(g.FIELDS)

for row in itertools.islice(g.iter_customers(), 10):
    print(row)

for row in itertools.islice(g.iter_accounts(), 10):
    print(row)

for row in itertools.islice(g.iter_devices(), 10):
```

```
    print(row)

for row in itertools.islice(g.iter_account_devices(), 10):
    print(row)

for row in itertools.islice(g.iter_transfers(), 10):
    print(row)
```

Each call to an iterator creates a fresh traversal of its stream. Therefore, previewing ten records did not require storing the complete stream or consuming the data intended for capture.

Source streams and expected record counts

The dataset contained five streams representing customers, accounts, devices, account-device observations, and money transfers.

Stream	PostgreSQL table	Expected rows
Customers	customers	228,000
Accounts	accounts	248,000
Devices	devices	228,600
Account-device observations	account_devices	285,600
Transfers	transfers	20,003,300
Total		**20,993,500**

The records represented activity during September 2026. Transfer timestamps were Unix seconds, and transfer amounts were integer Rwandan francs (RWF).

Device observation intervals used inclusive day numbers from 0 to 29, with day 0 representing 1 September 2026. The observations field represented the number of observations recorded during the interval.

Transfers were not globally time-sorted. Consequently, the capture program could not assume that the first or last record in the file represented the earliest or latest transaction.

Progressive CSV capture and audit

The capture.py program was implemented to write the five streams progressively using csv.DictWriter. It wrote the header before processing records and wrote each row as it was consumed.

The program maintained separate consumed and written counters, reported progress for every million transfers, measured capture duration, and recorded CSV sizes.

For the transfer stream, it also accumulated:

- Number of transfers.
- Earliest occurred_at.
- Latest occurred_at.
- Sum of amount_rwf.

The complete capture results were saved in capture_audit.json.

The program did not load all 20,993,500 source records into memory. This approach reduced memory requirements and allowed the large transfer stream to be written incrementally.

Verified capture results

Stream	Expected rows	Actual written rows	Result
Customers	228,000	228,000	PASS
Accounts	248,000	248,000	PASS
Devices	228,600	228,600	PASS
Account-device observations	285,600	285,600	PASS
Transfers	20,003,300	20,003,300	PASS
Total	**20,993,500**	**20,993,500**	**PASS**

The final capture verification returned:

ALL COUNTS MATCH: True

The measured capture duration was **168.777 seconds**.

The total size of the five CSV files was **798,833,384 bytes**, equivalent to **798.833384 MB** using decimal units.

The transfer file alone occupied **780,197,845 bytes**.

Transfer audit results

Audit measurement	Verified result
Transfer count	20,003,300
Earliest Unix timestamp	1,788,235,200
Latest Unix timestamp	1,790,798,399
Earliest time in Kigali	1 September 2026, 06:00:00
Latest time in Kigali	30 September 2026, 21:59:59
Total transfer amount	231,482,327,870 RWF

The transfer audit provides a baseline for validating the data after loading it into PostgreSQL and Neo4j. Agreement with this baseline indicates that the transfer count, time range, and total amount have been preserved.

The unchanged generator is imported as g. g.dataset_info() and g.FIELDS were inspected and ten records per stream were previewed with itertools.islice (fresh iterator calls restart each stream). Rows are written progressively with csv.DictWriter, with separate consumed and written counters, progress logged every million transfers, and an audit of transfer count, earliest/latest occurred_at and sum of amount_rwf.

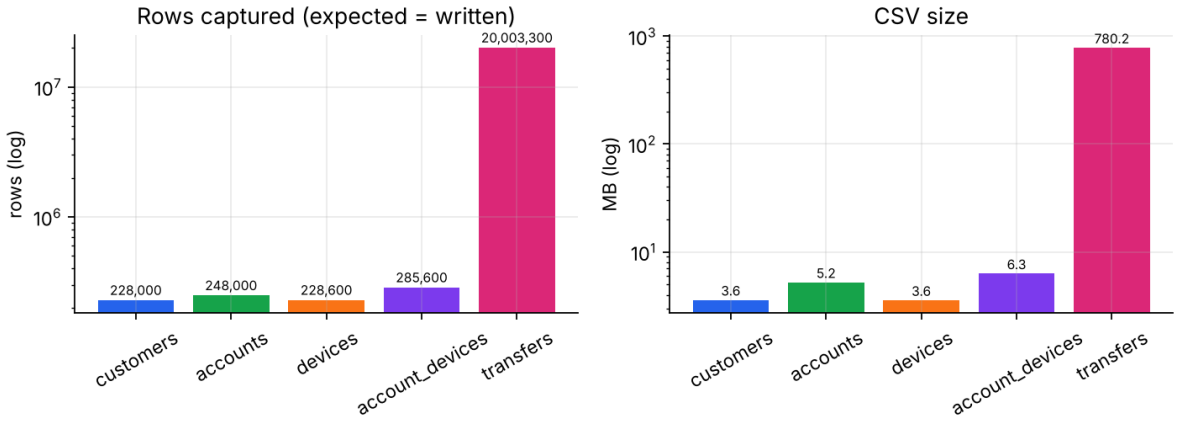


Figure 2. Rows captured and CSV size per stream (log scales).

Stream	Expected	Written	CSV size (MB)	Match
customers	228,000	228,000	3.582	Yes
accounts	248,000	248,000	5.213	Yes
devices	228,600	228,600	3.555	Yes
account_devices	285,600	285,600	6.284	Yes
transfers	20,003,300	20,003,300	780.198	Yes
Total	20,993,500	20,993,500	798.833	ALL COUNTS MATCH: True

Transfer audit	Result
Capture time	168.777 s (about 2 min 49 s)
Transfer count	20,003,300
Earliest / latest Unix second	1,788,235,200 / 1,790,798,399
Earliest / latest Kigali time	1 Sep 2026 06:00:00 / 30 Sep 2026 21:59:59
Sum of amount_rwf	231,482,327,870 RWF

2. Build PostgreSQL

2.1 Definition of the five tables. (Pk= Primary Key)

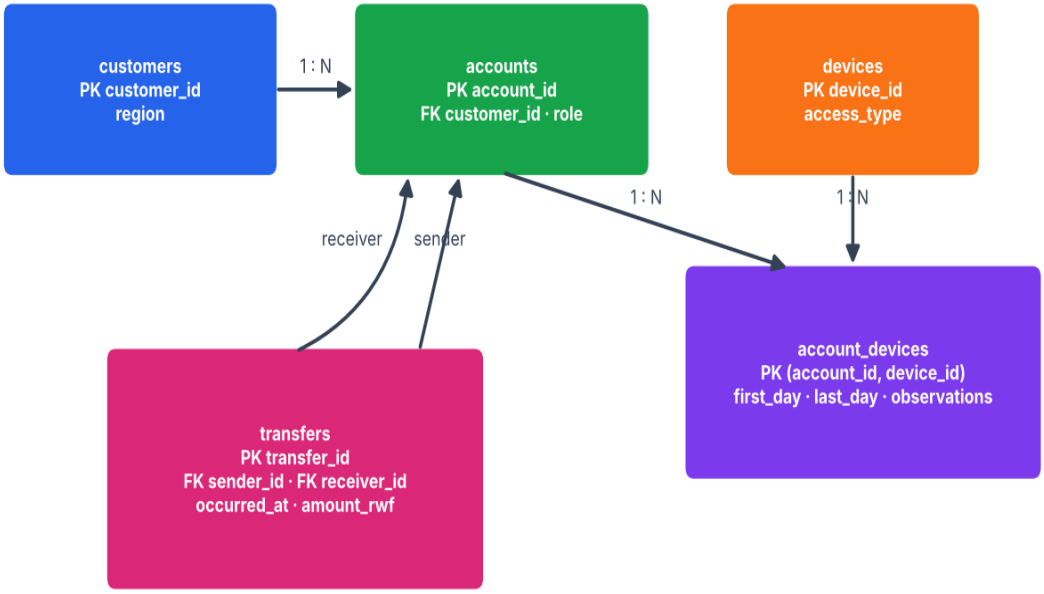


Figure 3. Relational schema (PK = primary key, FK = foreign key).

Table	Columns and units	Constraints
customers	customer_id INTEGER, region TEXT	PK customer_id
devices	device_id INTEGER, access_type TEXT (e.g. handset)	PK device_id
accounts	account_id INTEGER, customer_id INTEGER, role TEXT (person/merchant/agent)	PK account_id; FK customer_id → customers
account_devices	account_id, device_id INTEGER; first_day, last_day SMALLINT (day 0–29, 0 = 1 Sep); observations INTEGER (count)	PK (account_id, device_id); FKs to accounts and devices; observations > 0; 0 ≤ first_day ≤ last_day ≤ 29
transfers	transfer_id BIGINT; sender_id, receiver_id INTEGER; occurred_at BIGINT (Unix seconds); amount_rwf INTEGER (RWF)	PK transfer_id; FKs sender_id, receiver_id → accounts; amount_rwf > 0

Every column is NOT NULL. Inspection of pg_constraint returned 13 validated constraints: 5 primary keys, 5 foreign keys and 3 checks. The schema was applied with psql -f schema.sql.

1. Relational database design

The captured data was represented using five PostgreSQL tables in the database mobile_money_investigations.

The schema separated customers, accounts, devices, device observations, and transfers. This structure reduced unnecessary duplication and allowed relationships between entities to be enforced using foreign keys.

The database was created and the schema was applied using:

psql -U postgres -p 5433 -d mobile_money_investigations -f schema.sql

The command successfully created all five tables.

Table 1:customers

```
CREATE TABLE customers (
   customer_id INTEGER NOT NULL,
   region TEXT NOT NULL,
   CONSTRAINT customers_pkey
      PRIMARY KEY (customer_id)
);
```

The customers table stores customer identifiers and their regions.

- customer_id: unique integer identifying a customer.
- region: text identifying the customer's region.

The primary key prevents two customer records from having the same identifier.

Table 2: devices

```
CREATE TABLE devices (
   device_id INTEGER NOT NULL,
   access_type TEXT NOT NULL,
   CONSTRAINT devices_pkey
      PRIMARY KEY (device_id)
);
```

The devices table stores device identifiers and device access categories.

- device_id: unique integer identifying a device.
- access_type: text describing the type of device access, such as handset.

Table 3: accounts

```
CREATE TABLE accounts (
   account_id INTEGER NOT NULL,
   customer_id INTEGER NOT NULL,
   role TEXT NOT NULL,
   CONSTRAINT accounts_pkey
      PRIMARY KEY (account_id),
   CONSTRAINT accounts_customer_fk
      FOREIGN KEY (customer_id)
      REFERENCES customers (customer_id)
);
```

The accounts table stores accounts and their owners.

- account_id: unique account identifier.
- customer_id: identifier of the customer who owns the account.
- role: account category, such as person, merchant, or agent.

The foreign key ensures that an account cannot reference a customer who does not exist in the customers table.

Table 4: account_devices

```
CREATE TABLE account_devices (
   account_id INTEGER NOT NULL,
   device_id INTEGER NOT NULL,
   first_day SMALLINT NOT NULL,
   last_day SMALLINT NOT NULL,
   observations INTEGER NOT NULL,

   CONSTRAINT account_devices_pkey
      PRIMARY KEY (account_id, device_id),

   CONSTRAINT account_devices_account_fk
      FOREIGN KEY (account_id)
      REFERENCES accounts (account_id),

   CONSTRAINT account_devices_device_fk
      FOREIGN KEY (device_id)
      REFERENCES devices (device_id),

   CONSTRAINT account_devices_observations_positive
      CHECK (observations > 0),

   CONSTRAINT account_devices_days_valid
      CHECK (
         first_day >= 0
```

```
        AND first_day <= last_day
        AND last_day <= 29
    )
);
```

This table records observed relationships between accounts and devices.

- account_id: account associated with the device.
- device_id: observed device.
- first_day: first day of the observation interval.
- last_day: last day of the observation interval.
- observations: number of observations in that interval.

The primary key consists of both account_id and device_id. This prevents duplicate account-device pairs.

The day constraint guarantees that the interval is valid for the 30-day September dataset. The observations constraint ensures that the number of observations is positive.

Table 5:transfers

```
CREATE TABLE transfers (
  transfer_id BIGINT NOT NULL,
  sender_id INTEGER NOT NULL,
  receiver_id INTEGER NOT NULL,
  occurred_at BIGINT NOT NULL,
  amount_rwf INTEGER NOT NULL,

  CONSTRAINT transfers_pkey
    PRIMARY KEY (transfer_id),

  CONSTRAINT transfers_sender_fk
    FOREIGN KEY (sender_id)
    REFERENCES accounts (account_id),

  CONSTRAINT transfers_receiver_fk
    FOREIGN KEY (receiver_id)
    REFERENCES accounts (account_id),

  CONSTRAINT transfers_amount_positive
    CHECK (amount_rwf > 0)
);
```

The transfers table records individual money-transfer events.

- transfer_id: unique transaction identifier.
- sender_id: account sending the money.

- receiver_id: account receiving the money.
- occurred_at: Unix timestamp in seconds.
- amount_rwf: transfer amount in Rwandan francs.

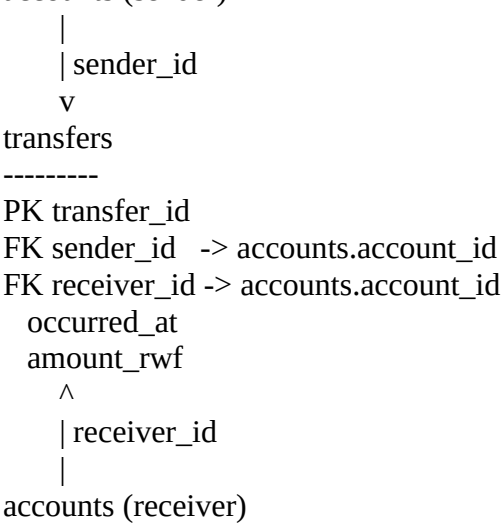
The two foreign keys enforce the existence of both endpoints. The primary key ensures that each transaction row has its own transfer identifier. The check constraint prevents zero or negative transfer amounts.

BIGINT was used for transfer IDs and timestamps to support their numerical range. INTEGER was used for entity IDs, amounts, and observation counts, while SMALLINT was used for day numbers.

Schema relationships and constraints

The relational schema can be represented as follows:

```
customers
---------
PK customer_id
  region
     |
     | 1 customer to many accounts
     v
accounts
--------
PK account_id
FK customer_id -> customers.customer_id
  role
     |
     | 1 account to many account-device records
     v
account_devices
---------------
PK (account_id, device_id)
FK account_id -> accounts.account_id
FK device_id  -> devices.device_id
  first_day
  last_day
  observations
     |
     v
devices
-------
PK device_id
  access_type
```



This structure supports both entity integrity and referential integrity. Primary keys uniquely identify records, foreign keys maintain valid relationships, and check constraints enforce the required numeric conditions.

Constraint validation

A constraint inspection query was used to confirm that all constraints were validated:

```
SELECT
   conname,
   contype,
   convalidated
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace
ORDER BY conname;
```

The recorded result contained **13 validated constraints**:

- Five primary keys.
- Five foreign keys.
- Three check constraints.

All 13 constraints were validated.

2.2 Bulking load and checking

Group 3_ Mobile Money Investigations with PostgreSQL and Neo4j_project_report

Bulk-loading the five CSV files

The data was loaded using PostgreSQL's \copy command. This command reads files from the client machine and transfers their contents to the database.

The loading order placed customers and devices before tables that referenced them. Accounts were loaded after customers, account-device observations after accounts and devices, and transfers after accounts.

The commands followed this pattern:

\copy customers FROM 'D:/Mobile_money_project_2/output/customers.csv' CSV HEADER;

\copy devices FROM 'D:/Mobile_money_project_2/output/devices.csv' CSV HEADER;

\copy accounts FROM 'D:/Mobile_money_project_2/output/accounts.csv' CSV HEADER;

\copy account_devices FROM 'D:/Mobile_money_project_2/output/account_devices.csv' CSV HEADER;

\copy transfers FROM 'D:/Mobile_money_project_2/output/transfers.csv' CSV HEADER;

Loaded row counts

Table	Loaded rows
customers	228,000
devices	228,600
accounts	248,000
account_devices	285,600
transfers	20,003,300
Total	**20,993,500**

Index creation and analysis

Indexes were created to improve the performance of ownership lookups, device-based investigations, and time-filtered transfer searches.

The following indexes were created:

```
CREATE INDEX idx_accounts_customer_account
ON accounts (customer_id, account_id);

CREATE INDEX idx_account_devices_device_account
ON account_devices (device_id, account_id);
```

```
CREATE INDEX idx_transfers_sender_time
ON transfers (sender_id, occurred_at)
INCLUDE (receiver_id, amount_rwf, transfer_id);

CREATE INDEX idx_transfers_receiver_time
ON transfers (receiver_id, occurred_at)
INCLUDE (sender_id, amount_rwf, transfer_id);

CREATE INDEX idx_transfers_occurred_at_brin
ON transfers USING BRIN (occurred_at);

ANALYZE;
```

The account index supports retrieving accounts by owner. The account-device index supports searching for accounts associated with a particular device.

The sender-time index supports queries that retrieve outgoing transfers for an account within a time range. The receiver-time index supports equivalent investigations of incoming transfers.

The BRIN index provides a compact index structure for timestamp filtering. Although the transfer stream was not globally time-sorted, the index can still help when physical data locality provides useful correlation with timestamps.

ANALYZE updated PostgreSQL's table statistics so that the query planner could estimate row counts and select appropriate execution plans.

Measured index setup times

Operation	Measured duration
idx_accounts_customer_account	244.091 ms
idx_account_devices_device_account	257.080 ms
idx_transfers_sender_time	34.816811 s
idx_transfers_receiver_time	38.764854 s
idx_transfers_occurred_at_brin	3.905760 s
ANALYZE	3.250474 s

These durations were measured during project execution. They describe the recorded index and analysis operations, not the CSV loading time.

Verify row counts, account roles, and transfer totals

The role distribution was checked with:

```
SELECT
   role,
   COUNT(*) AS account_count
FROM accounts
```

```
GROUP BY role
ORDER BY role;
```

The actual results were:

Account role	Count
Agent	2,000
Merchant	6,000
Person	240,000
Total	**248,000**

The results matched the expected distribution.

The transfer audit was checked using:

```
SELECT
   COUNT(*) AS transfer_count,
   MIN(occurred_at) AS earliest_unix,
   MAX(occurred_at) AS latest_unix,
   SUM(amount_rwf::BIGINT) AS total_amount_rwf,
   TO_TIMESTAMP(MIN(occurred_at))
      AT TIME ZONE 'Africa/Kigali' AS earliest_kigali,
   TO_TIMESTAMP(MAX(occurred_at))
      AT TIME ZONE 'Africa/Kigali' AS latest_kigali
FROM transfers;
```

The cast to BIGINT before summation prevents overflow from accumulating the amounts in a 32-bit integer.

The verified PostgreSQL output was:

Audit field	PostgreSQL result
Transfer count	20,003,300
Earliest Unix timestamp	1,788,235,200
Latest Unix timestamp	1,790,798,399
Total amount	231,482,327,870 RWF
Earliest Kigali time	2026-09-01 06:00:00
Latest Kigali time	2026-09-30 21:59:59

These values matched the original capture audit.

Constraint and index verification

The constraint query confirmed 13 validated constraints. The index catalog query returned ten indexes: five explicitly created indexes and five primary-key indexes.

The relevant catalog query was:

```
SELECT
   indexname,
   indexdef
FROM pg_indexes
WHERE schemaname = 'public'
ORDER BY indexname;
```

The results confirmed the presence of the required indexes.

Database and index storage sizes

All storage measurements use decimal units, where **1 GB = 1,000,000,000 bytes**.

Table	Table size (GB)	Index size (GB)	Total (GB)
customers	0.010265	0.005136	0.015401
devices	0.010265	0.005153	0.015417
accounts	0.011313	0.011182	0.022495
account_devices	0.012878	0.012796	0.025674
transfers	1.205699	2.433499	3.639198

The total database size reported by PostgreSQL was **3,726,482,455 bytes**, equivalent to **3.726482455 GB**.

The transfer indexes occupied approximately twice the space of the transfer table itself. This reflects the storage cost of supporting multiple access patterns on a large transaction table.

The total database size can differ from the sum of the table-level figures because database-wide storage includes additional objects and overhead.

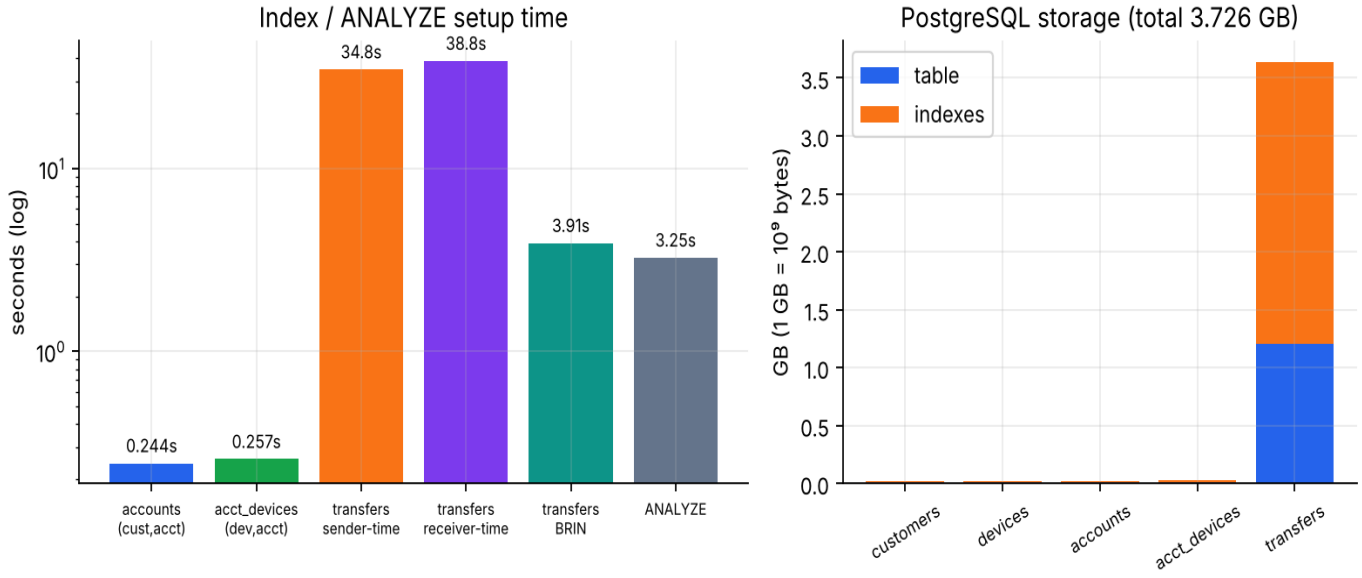


Figure 4. Index/ANALYZE setup time (left) and PostgreSQL table vs index storage (right).

Check	PostgreSQL result	Matches capture
Row counts (5 tables)	228,000 / 228,600 / 248,000 / 285,600 / 20,003,300	Yes
Account roles	person 240,000; merchant 6,000; agent 2,000	Yes
Transfer audit	20,003,300; 1,788,235,200–1,790,798,399; 231,482,327,870 RWF	Yes
Database size	3,726,482,455 bytes = 3.726 GB (transfer indexes 2.433 GB vs table 1.206 GB)	—

3. Build Neo4j

3.1 Mapping the same facts

Graph data model

The same source records were represented in Neo4j as a property graph. In this model, customers, accounts, and devices are nodes, while ownership, device observations, and money transfers are relationships.

The graph schema contains three node labels:

Customer

```
(:Customer {
   customer_id: INTEGER,
   region: STRING
})
```

Account

```
(:Account {
   account_id: INTEGER,
   role: STRING
})
```

Device

```
(:Device {
   device_id: INTEGER,
   access_type: STRING
})
```

The graph contains three relationship types.

Ownership

```
(:Customer)-[:OWNS]->(:Account)
```

Device usage

```
(:Account)-[:USED_DEVICE {
   first_day: INTEGER,
   last_day: INTEGER,
   observations: INTEGER
}]->(:Device)
```

Money transfer

```
(:Account)-[:TRANSFER {
   transfer_id: LONG,
   occurred_at: LONG,
   amount_rwf: INTEGER
}]->(:Account)
```

The OWNS relationship represents ownership, while USED_DEVICE preserves the observed relationship between an account and a device. The transfer relationship is directed from the sending account to the receiving account.

2. Mapping source fields to the graph

Source table and field	Graph destination
customers.customer_id	Customer.customer_id
customers.region	Customer.region
accounts.account_id	Account.account_id
accounts.role	Account.role
accounts.customer_id	OWNS relationship from the customer to the account
devices.device_id	Device.device_id
devices.access_type	Device.access_type
account_devices.account_id	Start node of USED_DEVICE
account_devices.device_id	End node of USED_DEVICE
account_devices.first_day	USED_DEVICE.first_day
account_devices.last_day	USED_DEVICE.last_day
account_devices.observations	USED_DEVICE.observations
transfers.transfer_id	TRANSFER.transfer_id
transfers.sender_id	Start account of TRANSFER
transfers.receiver_id	End account of TRANSFER
transfers.occurred_at	TRANSFER.occurred_at
transfers.amount_rwf	TRANSFER.amount_rwf

The customer_id field in the source accounts CSV is necessary for relational ownership, but Neo4j import was configured to ignore that column in the Account node file. Ownership was instead represented explicitly through OWNS relationships.

Preserving repeated payments

Every transfer was imported as a separate TRANSFER relationship, even when the same sender and receiver exchanged money multiple times.

For example:

```
(Account 1)-[:TRANSFER {
   transfer_id: 101,
   occurred_at: 1788246429,
   amount_rwf: 22800
}]->(Account 25717)

(Account 1)-[:TRANSFER {
   transfer_id: 102,
   occurred_at: 1788246500,
   amount_rwf: 15000
}]->(Account 25717)
```

These two relationships illustrate the required representation of separate events; the example values are illustrative, not an additional claim about the imported records.

Collapsing repeated payments into a single relationship would discard the distinction between individual events. It could remove transaction identifiers, individual amounts, and event timestamps needed to reconstruct payment sequences, investigate suspicious activity, and establish when money moved.

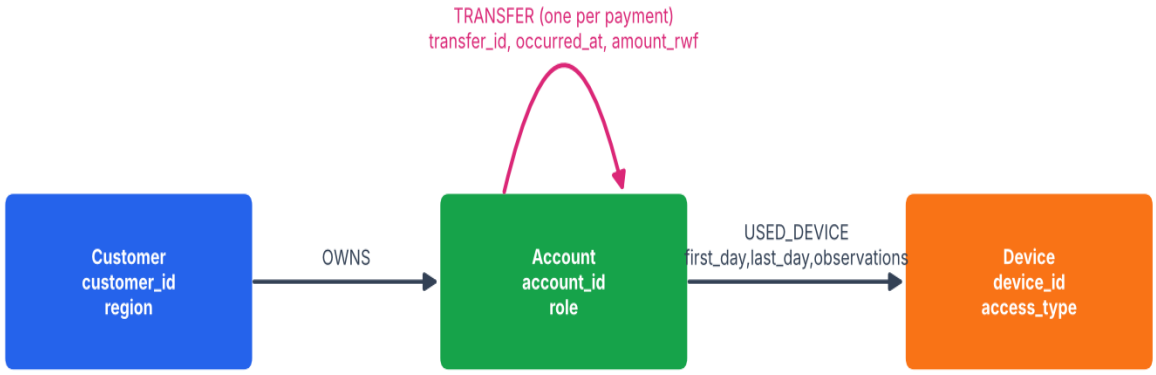


Figure 5. Graph schema: three node labels and three relationship types.

Source field	Graph destination
customers.customer_id, region	Customer.customer_id, region
accounts.account_id, role	Account.account_id, role
accounts.customer_id	(Customer)-[:OWNS]->(Account)
devices.device_id, access_type	Device.device_id, access_type
account_devices.account_id / device_id	start / end node of USED_DEVICE
first_day, last_day, observations	USED_DEVICE properties
transfers.sender_id / receiver_id	start / end Account of TRANSFER
transfer_id, occurred_at, amount_rwf	TRANSFER properties

Each transfer is a separate TRANSFER relationship, so repeated payments between the same accounts are preserved.

3.2 Preparation and importing CSVs

Create prepare_import.py

The script prepare_import.py generated temporary import copies with Neo4j's typed CSV headers. The original files in output were left unchanged.

The script processed the source files progressively rather than loading the entire transfer file into memory.

The temporary import files were:

Import file	Purpose
customers.csv	Customer nodes
accounts.csv	Account nodes
devices.csv	Device nodes
owns.csv	Customer-to-account ownership
used_device.csv	Account-to-device relationships
transfers.csv	Individual money-transfer relationships

The generated headers were:

customers.csv:
customer_id:ID(Customer),region:string
accounts.csv:
account_id:ID(Account),:IGNORE,role:string
devices.csv:
device_id:ID(Device),access_type:string
owns.csv:
:START_ID(Customer),:END_ID(Account)
used_device.csv:
:START_ID(Account),:END_ID(Device),first_day:int,last_day:int,observations:int
transfers.csv:
transfer_id:long,:START_ID(Account),:END_ID(Account),occurred_at:long,amount_rwf:in

The :ID(Customer), :ID(Account), and :ID(Device) headers defined separate ID groups. This allowed integer identifiers from different entity categories to remain distinct.

The :IGNORE column retained the source customer ID in the account file but prevented Neo4j from importing it as an Account node property. The ownership relationship was imported separately through owns.csv.

The preparation checks confirmed that all generated import row counts matched the expected source counts.

Temporary file sizes

Temporary import file	Size (MB, decimal)

Customers	3.354
Accounts	4.965
Devices	3.326
Ownership relationships	3.219
Device relationships	5.999
Transfers	760.195

The temporary transfer file was the largest because it contained more than 20 million individual transaction relationships.

2. Stop the empty database and perform the full import

Before the import, the Neo4j database was checked to ensure that it contained no existing nodes or relationships. The counts were zero.

The running Neo4j container was then stopped:

```
docker stop mobile-money-neo4j
```

The full import was executed using:

```
docker run --rm --user neo4j ^
  -v mobile_money_neo4j_data:/data ^
  -v "D:\Mobile_money_project_2\neo4j_import:/var/lib/neo4j/import:ro" ^
  neo4j:2026.09.0 ^
  neo4j-admin database import full neo4j ^
  --overwrite-destination=true ^
  --id-type=integer ^
  --nodes=Customer=/var/lib/neo4j/import/customers.csv ^
  --nodes=Account=/var/lib/neo4j/import/accounts.csv ^
  --nodes=Device=/var/lib/neo4j/import/devices.csv ^
  --relationships=OWNS=/var/lib/neo4j/import/owns.csv ^
  --relationships=USED_DEVICE=/var/lib/neo4j/import/used_device.csv ^
  --relationships=TRANSFER=/var/lib/neo4j/import/transfers.csv
```

The command used the temporary import copies mounted read-only. The --id-type=integer option ensured integer identifiers were used for node IDs.

The import completed successfully. Neo4j reported:

```
Post processing COMPLETED in 8s 646ms.
IMPORT DONE in 4m 23s 340ms.

Imported:
  704600 nodes
  20536900 relationships
```

62275900 properties

Peak memory usage: 555.7MiB

The measured full-import duration was **4 minutes, 23.340 seconds**.

The reported import relationship total was 20,536,900, which matched the sum of the three source relationship counts.

3. Start Neo4j and configure constraints and indexes

After the import, the database container was restarted:

docker start mobile-money-neo4j

Three uniqueness constraints were created for the business identifiers:

```
CREATE CONSTRAINT customer_id_unique
FOR (c:Customer)
REQUIRE c.customer_id IS UNIQUE;

CREATE CONSTRAINT account_id_unique
FOR (a:Account)
REQUIRE a.account_id IS UNIQUE;

CREATE CONSTRAINT device_id_unique
FOR (d:Device)
REQUIRE d.device_id IS UNIQUE;
```

These constraints ensure that a customer, account, or device cannot have multiple nodes with the same corresponding business ID.

A range index was created for transfer timestamps:

```
CREATE RANGE INDEX transfer_occurred_at_idx
FOR ()-[t:TRANSFER]-()
ON (t.occurred_at);
```

The index was initially observed in the POPULATING state and later reached ONLINE with 100% population.

This index supports transfer searches based on occurred_at.

Neo4j node and relationship counts

The imported node counts were verified:

Node label	Actual count
Customer	228,000
Account	248,000
Device	228,600
Total nodes	**704,600**

The relationship counts were verified:

Relationship type	Actual count
OWNS	248,000
USED_DEVICE	285,600
TRANSFER	20,003,300
Total relationships	**20,536,900**

These results matched the source data and the assignment requirements.

Account role verification

The following Cypher query checked the distribution of account roles:

```
MATCH (a:Account)
RETURN
   a.role AS account_role,
   count(*) AS account_count
ORDER BY account_role;
```

The results were:

Account role	Count
Agent	2,000
Merchant	6,000
Person	240,000
Total	**248,000**

The graph role counts matched PostgreSQL.

Ownership verification

The following query checked whether each account had exactly one incoming ownership relationship:

```
MATCH (a:Account)
OPTIONAL MATCH (c:Customer)-[r:OWNS]->(a)
WITH a, count(r) AS owner_relationships
RETURN
   count(a) AS total_accounts,
```

```
    sum(
      CASE
        WHEN owner_relationships = 1 THEN 1
        ELSE 0
      END
    ) AS accounts_with_exactly_one_owner,
    sum(
      CASE
        WHEN owner_relationships = 0 THEN 1
        ELSE 0
      END
    ) AS accounts_without_owner,
    sum(
      CASE
        WHEN owner_relationships > 1 THEN 1
        ELSE 0
      END
    ) AS accounts_with_multiple_owners;
```

The actual result was:

Check	Result
Total accounts	248,000
Accounts with exactly one owner	248,000
Accounts without an owner	0
Accounts with multiple owners	0

This confirmed that every imported account had exactly one OWNS relationship.

Transfer audit verification

The transfer count, total amount, and time range were checked in Neo4j:

```
MATCH ()-[t:TRANSFER]->()
RETURN
  count(t) AS total_transfers,
  sum(t.amount_rwf) AS total_amount_rwf,
  min(t.occurred_at) AS earliest_unix,
  max(t.occurred_at) AS latest_unix;
```

The results were:

Audit field	Neo4j result
Total transfers	20,003,300
Total amount	231,482,327,870 RWF
Earliest Unix timestamp	1,788,235,200

Latest Unix timestamp	1,790,798,399

These values matched both the capture audit and PostgreSQL.

An additional check confirmed that all 20,003,300 transfer relationships had a non-null transfer_id, with a minimum of 1 and maximum of 20,003,300.

Transfer-ID uniqueness verification remains incomplete. The all-at-once distinct-ID query exceeded Neo4j's transaction memory limit. Three manually checked batches covering IDs 1–300,000 reported zero duplicates, but the remaining IDs have not yet been checked. Therefore, this report does not claim that global uniqueness has been conclusively established in Neo4j.

Neo4j storage measurements

The measured Neo4j store and transaction-log sizes were obtained from the running container:

docker exec mobile-money-neo4j sh -c "du -s -B1 /data/databases/neo4j /data/transactions/neo4j 2>/dev/null"

The output was:

2171121664 /data/databases/neo4j
269488128 /data/transactions/neo4j

Using decimal GB:

Storage component	Bytes	Size (GB)
Database store	2,171,121,664	2.171122
Transaction logs	269,488,128	0.269488
Combined measured size	**2,440,609,792**	**2.440610**

The transaction-log size can change as Neo4j continues operating, so these values represent the time of measurement.

The full import duration was recorded as 263.340 seconds. A separate total setup duration, including all subsequent index population and verification operations, was not recorded.

The import finished in 4 min 23.340 s (263.340 s), with 704,600 nodes, 20,536,900 relationships, 62,275,900 properties and peak memory 555.7 MiB. After restart, unique constraints were created on customer_id, account_id and device_id, plus a range index on TRANSFER.occurred_at, which reached ONLINE (100%).

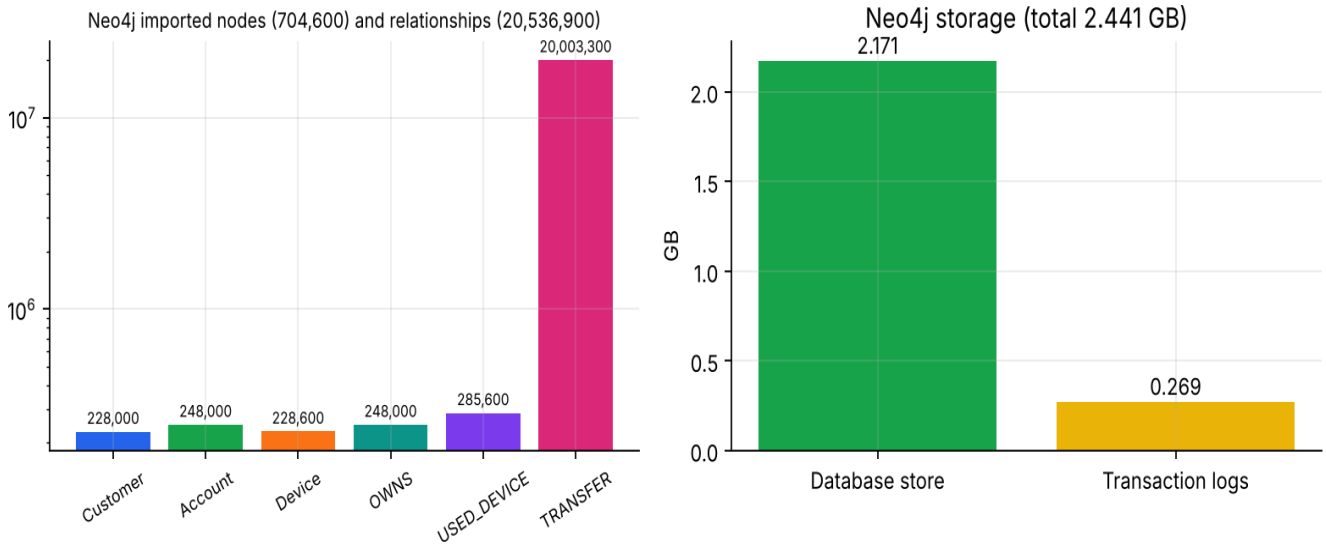


Figure 6. Neo4j imported counts (log scale) and store/transaction-log size.

Verification	Neo4j result	Status
Nodes	Customer 228,000; Account 248,000; Device 228,600 = 704,600	Match
Relationships	OWNS 248,000; USED_DEVICE 285,600; TRANSFER 20,003,300 = 20,536,900	Match
Roles	person 240,000; merchant 6,000; agent 2,000	Match
Ownership	248,000 accounts with exactly one owner; 0 without; 0 with several	Pass
Transfer audit	20,003,300; 231,482,327,870 RWF; 1,788,235,200–1,790,798,399	Match
Transfer-ID uniqueness	Python batch check over all 20,003,300 relationships: 0 duplicates, 0 extra, 0 missing (about 25.2 s); IDs 1–20,003,300	Pass
Storage	Store 2,171,121,664 B + logs 269,488,128 B = 2.441 GB	—

3.3 Inspecting one account in both models

To compare the relational and graph models, account 1 was selected from the loaded database. A PostgreSQL query joined the account to its customer, device, and one outgoing transfer.

```
SELECT
    a.account_id,
    a.customer_id,
    c.region,
    a.role,
    ad.device_id,
    d.access_type,
    t.transfer_id,
    t.receiver_id,
    t.occurred_at,
    t.amount_rwf
FROM accounts AS a
JOIN customers AS c
    ON c.customer_id = a.customer_id
JOIN account_devices AS ad
    ON ad.account_id = a.account_id
JOIN devices AS d
    ON d.device_id = ad.device_id
JOIN transfers AS t
    ON t.sender_id = a.account_id
LIMIT 1;
```

The query returned the following actual record:

Field	PostgreSQL value
account_id	1
customer_id	1
region	Kigali
role	person
device_id	1
access_type	handset
transfer_id	129973
receiver_id	25717
occurred_at	1788246429
amount_rwf	22800

The query took **235.374 milliseconds**.

2. Trace the same account using Cypher MATCH

The corresponding Neo4j query was:

```
MATCH (c:Customer)-[:OWNS]->(a:Account {account_id: 1})
MATCH (a)-[ud:USED_DEVICE]->(d:Device {device_id: 1})
MATCH (a)-[t:TRANSFER {transfer_id: 129973}]->(receiver:Account)
RETURN
   c.customer_id AS customer_id,
   c.region AS region,
   a.account_id AS sender_account,
   a.role AS role,
   d.device_id AS device_id,
   d.access_type AS access_type,
   t.transfer_id AS transfer_id,
   receiver.account_id AS receiver_id,
   t.occurred_at AS occurred_at,
   t.amount_rwf AS amount_rwf;
```

The Neo4j query returned the same ten field values as PostgreSQL:

Field	PostgreSQL	Neo4j	Match
Customer ID	1	1	Yes
Region	Kigali	Kigali	Yes
Sender account	1	1	Yes
Role	person	person	Yes
Device ID	1	1	Yes
Device access type	handset	handset	Yes
Transfer ID	129973	129973	Yes
Receiver account	25717	25717	Yes
Unix timestamp	1788246429	1788246429	Yes
Amount (RWF)	22,800	22,800	Yes

All ten values matched. This demonstrates that the ownership relationship, device association, and selected outgoing transfer were preserved during graph import for this example.

3. Explain foreign keys and graph relationships

In PostgreSQL, relationships are represented using foreign keys that reference primary keys in other tables. To retrieve connected information, a query uses JOIN operations.

For example, accounts.customer_id references customers.customer_id. The corresponding graph representation is a directed OWNS relationship from a Customer node to an Account node.

Similarly, the account_devices table contains account and device identifiers. In Neo4j, those references become an USED_DEVICE relationship, with the observation interval and count stored as relationship properties.

The transfers table references the account table twice: once for the sender and once for the receiver. In the graph, these endpoints become the start and end nodes of a directed TRANSFER relationship.

The two models represent the same underlying facts but organize them differently. PostgreSQL explicitly stores keys in tables and uses joins to navigate relationships. Neo4j stores the connections directly as relationships, making multi-step traversal natural.

Why repeated payments must remain separate

Each transfer represents a distinct event. If two accounts exchange money multiple times, each payment must retain its own transfer ID, timestamp, and amount.

Collapsing repeated payments into a single edge could remove the evidence needed to:

- Distinguish individual payments between the same accounts.
- Reconstruct the order and timing of transfers.
- Identify repeated transactions within a suspicious time window.
- Compare transaction amounts.
- Investigate the sequence of money movement between accounts.

Preserving separate relationships therefore supports temporal analysis and transaction-level investigations.

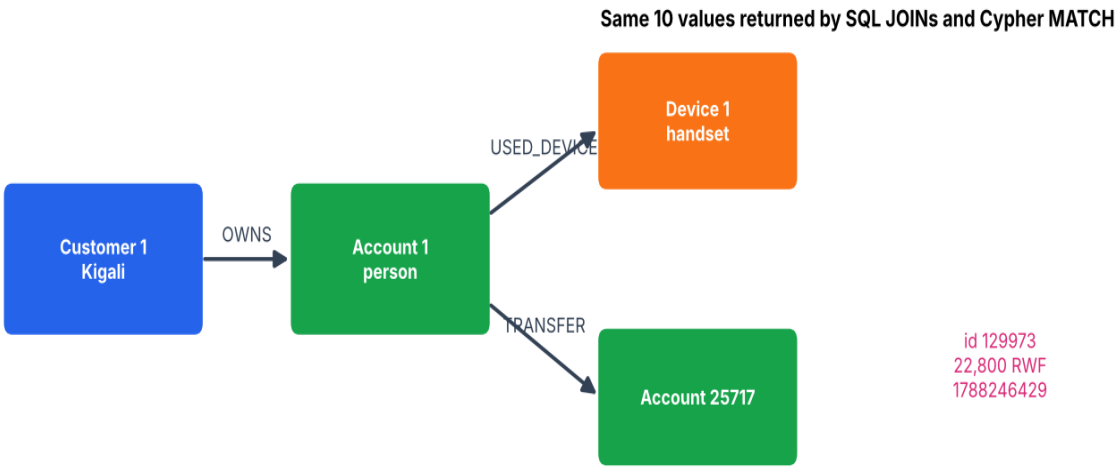


Figure 7. Account 1 traced to its owner, a device and an outgoing transfer.

A SQL query joining accounts, customers, account_devices, devices and transfers (0.235 s) and a Cypher MATCH over OWNS, USED_DEVICE and TRANSFER returned identical values for all ten fields: customer 1 (Kigali), account 1 (person), device 1 (handset), transfer 129973 to account 25717 at 1788246429 for 22,800 RWF.

Foreign keys become relationships: accounts.customer_id becomes OWNS, account_devices rows become USED_DEVICE with their interval properties, and the two account keys in transfers become the start and end of TRANSFER. Collapsing repeated payments into one edge would lose transfer IDs, individual amounts and timestamps, so sequences, chains and cycles could no longer be rebuilt.

4. Answer routine business questions

4.1 Q1 Account activity

For each review account the SQL and Cypher queries return outgoing count and total RWF (zero when no match, all receiver roles) between 09:45:00 inclusive and 10:45:00 exclusive CAT. Boundaries are converted to epoch seconds in Python.

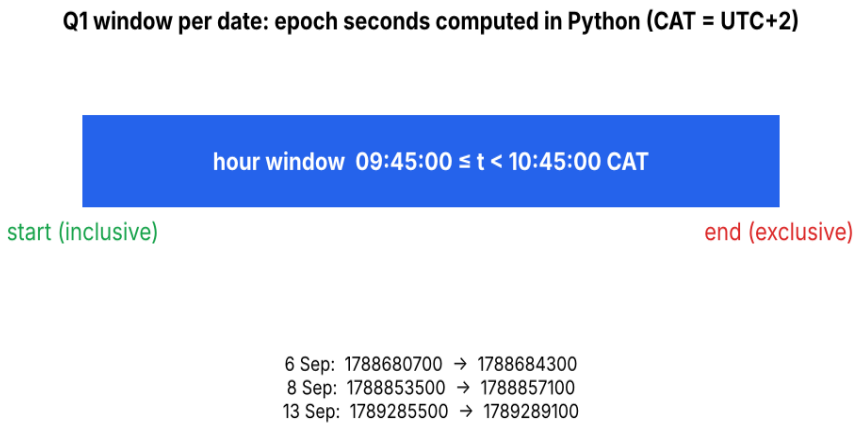


Figure 8. Q1 hour boundaries for the listed dates, as epoch seconds (CAT).

Outgoing value is not a balance: it counts only money sent in one hour, ignoring incoming money, earlier holdings, fees and cash-in or cash-out, so it says nothing about what the wallet holds.

4.2 Q2 Daily activity

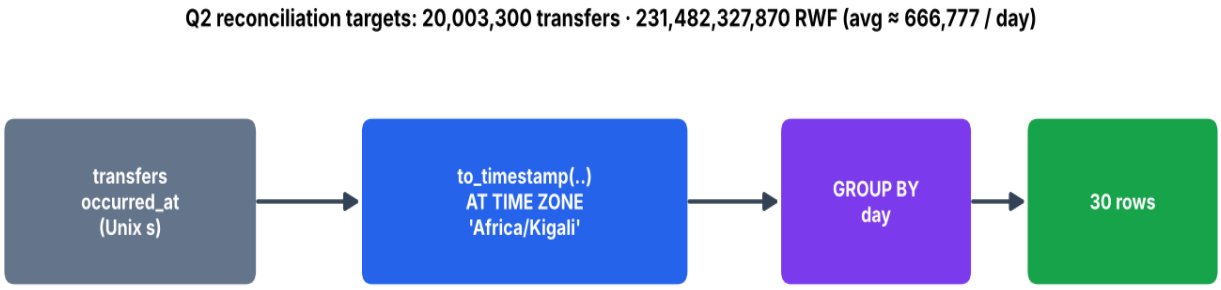


Figure 9. Q2 daily grouping pipeline and reconciliation targets.

```
mobile_money_investigations-# GROUP BY transfer_date
mobile_money_investigations-# ORDER BY transfer_date;
 transfer_date | transfer_count | total_rwf
---------------+----------------+-----------
 2026-09-01    |         666667 | 7683326000
 2026-09-02    |         666667 | 7678846900
 2026-09-03    |         666666 | 7665877900
 2026-09-04    |         666667 | 7665199000
 2026-09-05    |         666667 | 7679091900
 2026-09-06    |         666816 | 7741994180
 2026-09-07    |         666817 | 7726576180
 2026-09-08    |         666907 | 7777047990
 2026-09-09    |         666906 | 7783400210
 2026-09-10    |         666997 | 7807170170
 2026-09-11    |         666847 | 7746125100
 2026-09-12    |         666936 | 7783593400
 2026-09-13    |         666937 | 7800995510
 2026-09-14    |         666997 | 7856644775
 2026-09-15    |         666996 | 7859428355
 2026-09-16    |         666787 | 7683087200
 2026-09-17    |         666757 | 7699710100
 2026-09-18    |         666786 | 7677843700
 2026-09-19    |         666757 | 7665841400
 2026-09-20    |         666757 | 7663404900
 2026-09-21    |         666726 | 7696315800
 2026-09-22    |         666727 | 7703220100
 2026-09-23    |         666697 | 7697325900
 2026-09-24    |         666726 | 7668636000
 2026-09-25    |         666697 | 7677231000
 2026-09-26    |         666697 | 7671715700
 2026-09-27    |         666696 | 7692655900
 2026-09-28    |         666667 | 7676184700
 2026-09-29    |         666667 | 7668892500
 2026-09-30    |         666666 | 7684945400
(30 rows)
```

4.3 Q3 Shared devices

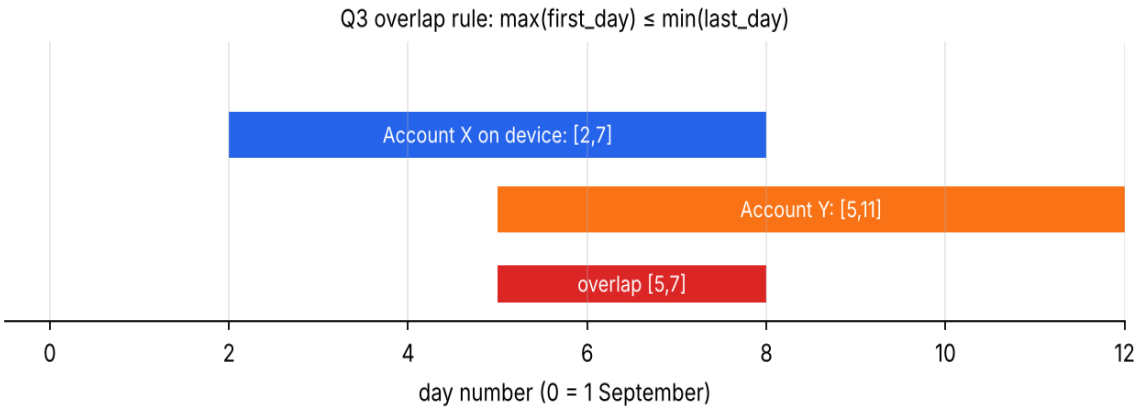


Figure 10. Q3 overlap rule using the intervals [2,7] and [5,11], which overlap on days [5,7].

5. Investigation of connected accounts

5.1 Q4 Ownership and device network

Distinct other accounts connected through OWNS or USED_DEVICE in either direction within 4, 6, 8 and 10 links, grouped by owner region. SQL uses a recursive walk over an edges CTE that carries node kind and ID; Cypher uses a bounded undirected path and DISTINCT accounts. All 24 SQL/Cypher comparisons (6 sets × 4 depths) passed.

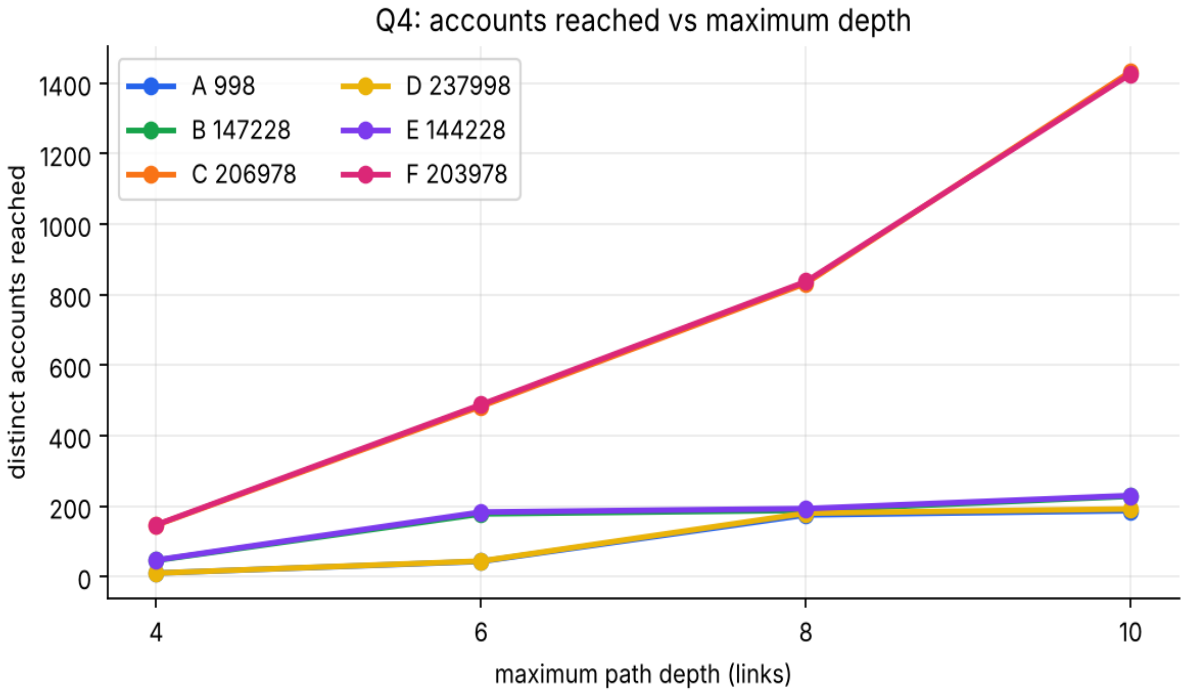


Figure 11. Q4 distinct accounts reached against maximum depth.

Set	Account	Depth 4	Depth 6	Depth 8	Depth 10
A	998	10	43	175	187
B	147,228	46	178	188	228
C	206,978	146	483	832	1,433
D	237,998	10	44	180	192
E	144,228	47	183	193	230
F	203,978	147	488	837	1,426

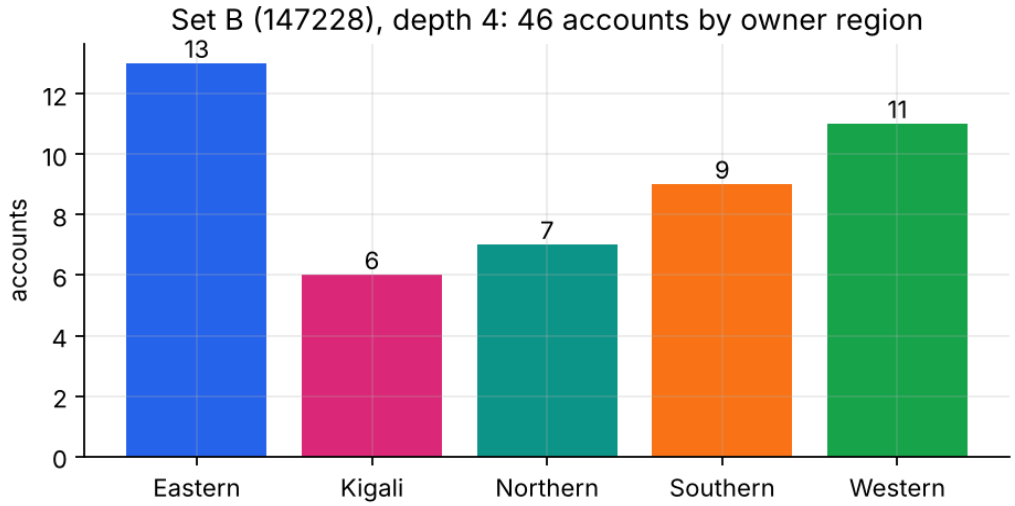


Figure 12. Example regional breakdown: set B, depth 4 (46 accounts).

Counts rise with depth in every case; C and F grow most (about 1,430 accounts at depth 10), A and D least. For F the step from depth 8 to 10 adds 589 accounts. Longer paths pass through more intermediate customers and devices, so they add indirect and weaker associations. A connection alone is not evidence of fraud, and a widely used device can link unrelated customers.

5.2 Q5 Rapid onward payments

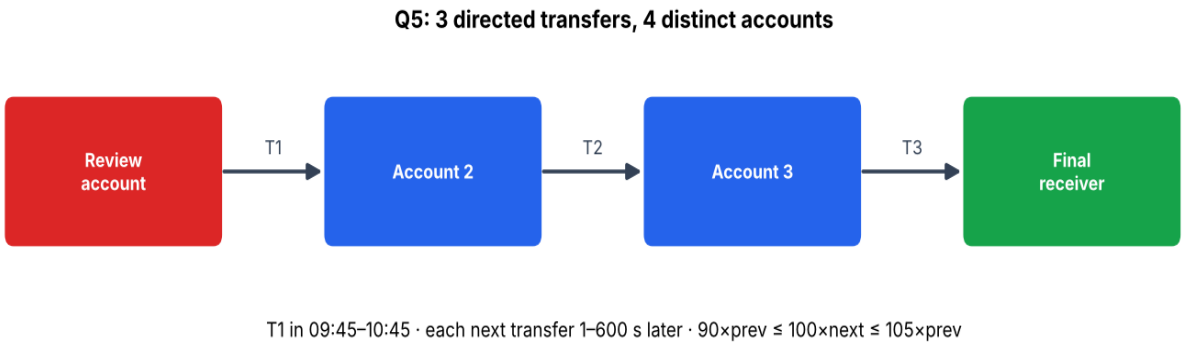
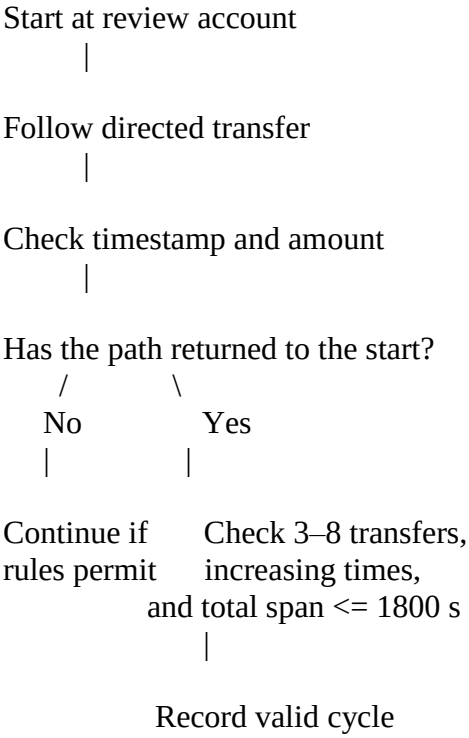


Figure 13. Q5 rule: three directed transfers through four distinct accounts.

SQL joins three transfer aliases (receiver to sender); Cypher matches three directed TRANSFER relationships. Rules: first transfer starts at the review account inside its hour; each next transfer is 1 to 600 s later; 90×previous ≤ 100×next ≤ 105×previous. Results are sorted by ordered transfer IDs and final receiver.

5.3 Q6 Circular payments

Cycle Validation Process



Performance Results

Review case	PostgreSQL mean time	Neo4j mean time
B	2.6636 ms	20.8958 ms
E	1.4934 ms	16.7874 ms

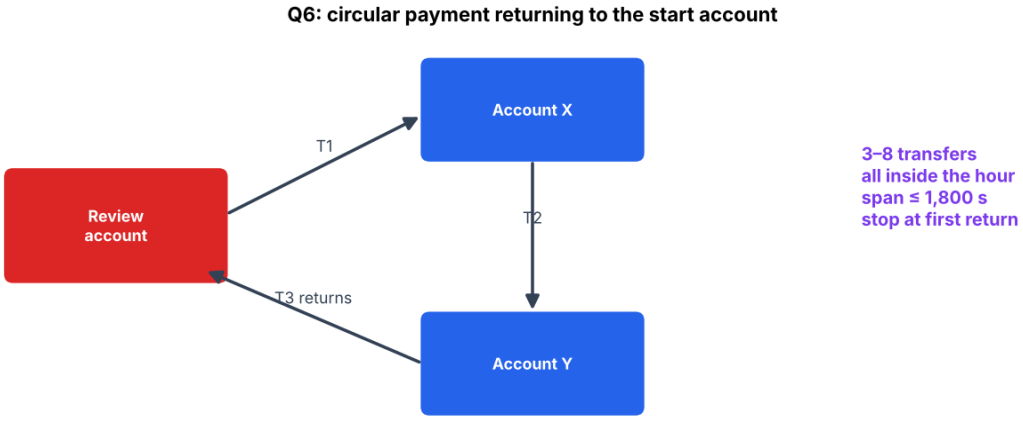


Figure 14. Q6 rule: a 3–8 transfer cycle back to the review account.

Cycles of 3 to 8 transfers start and end at the review account, lie inside its hour, are strictly time-ordered, span at most 1,800 s, keep the Q5 amount rule and stop at the first return. SQL uses a recursive query carrying current account, ordered IDs, first/last time and last amount; Cypher uses a bounded path with equivalent checks.

5.4 Q7 Shortest transfer connection

Performance Results

Workload	PostgreSQL mean time	Neo4j mean time
Case B, bound 4	1,568.8124 ms	42.1237 ms
Case B, bound 6	18,166.7615 ms	121.5205 ms
Case E, bound 4	1,620.5814 ms	46.7973 ms
Case E, bound 6	22,283.8590 ms	330.0907 ms

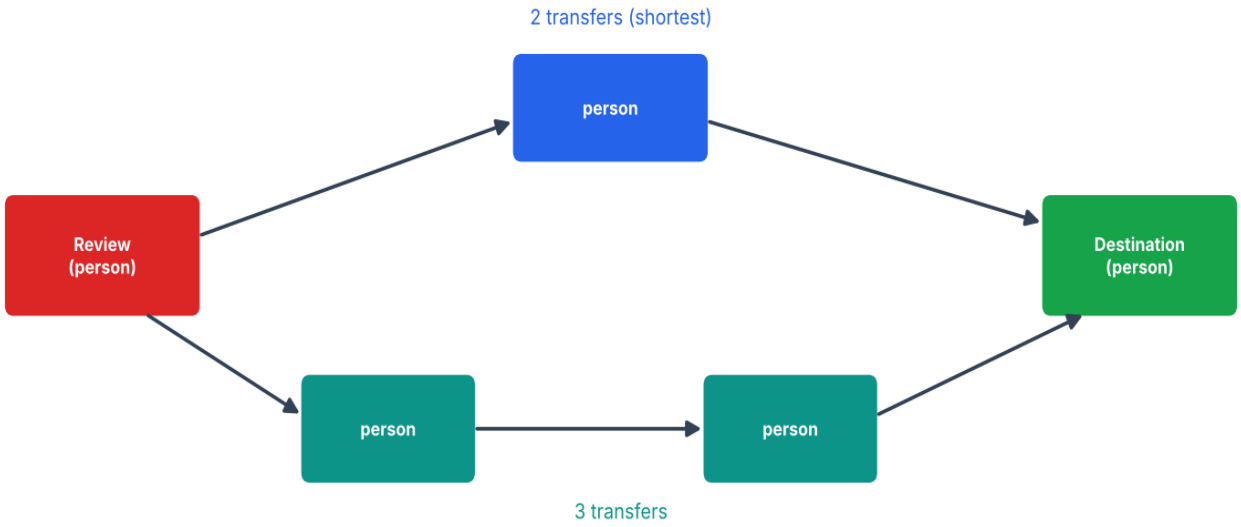


Figure 15. Q7 shortest directed transfer route through person accounts only.

The fewest directed transfers from each review account to its destination, with every account on the route a person, using bounds 4 and 6. The person restriction is applied during the search. SQL uses a recursive search; Neo4j uses native shortest-path search. One supporting path per successful case is retrieved and checked for IDs, direction, roles, event existence and length.

6. Check correctness and performance

6.1 Building one comparison program

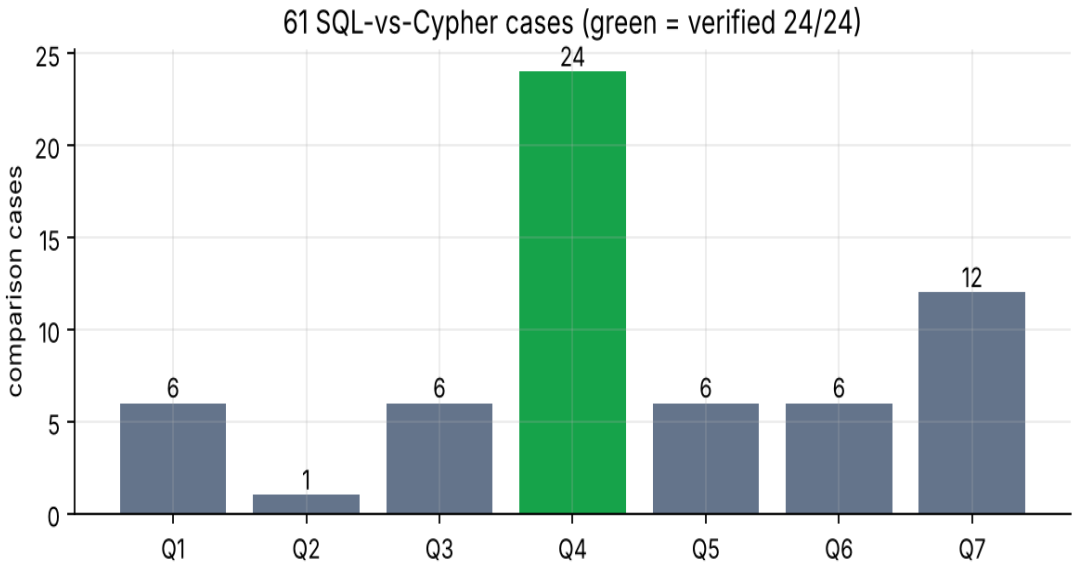


Figure 16. Comparison cases per question (61 in total).

run_pg(query, params) and run_neo(query, params) return every row; integers are normalised and rows sorted before exact comparison. Q4 passed 24 of 24. Q7 lengths are compared separately from supporting paths.

6.2 Time a fixed selection

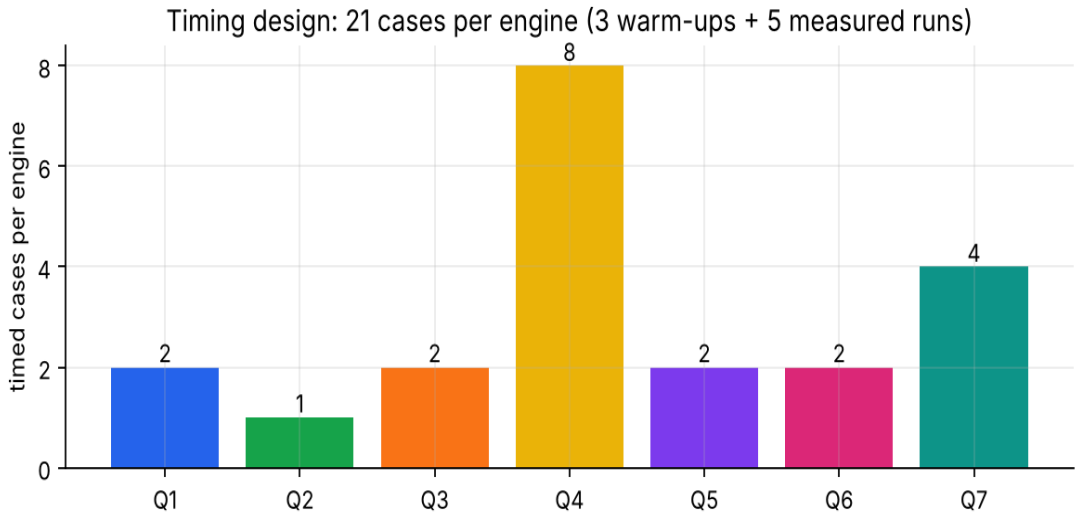


Figure 17. Timed cases per engine (21 in total).

Cases: Q1/Q3/Q5/Q6 for B and E, Q2 for the full month, Q4 for B and E at four depths, Q7 for B and E at both bounds; 3 untimed warm-ups and 5 measured runs each, timed with perf_counter, with errors and timeouts recorded separately.

6.3 Trying one SQL improvement

The recorded estimated PostgreSQL Q7 plan includes an index-only scan using idx_transfers_sender_time, nested-loop operations, primary-key account lookups, role filters, and sorting.

Simplified Q7 Query Execution Flow

Filter transfers by time and direction
|
v
Use sender/time index where applicable
|
v
Explore candidate directed paths
|
v
Look up account roles

|
v
Apply path-bound and role constraints
|
v
Select shortest qualifying route

The before-and-after SQL timing table and complete 12-case equivalence results are not available here. No improvement percentage is claimed.

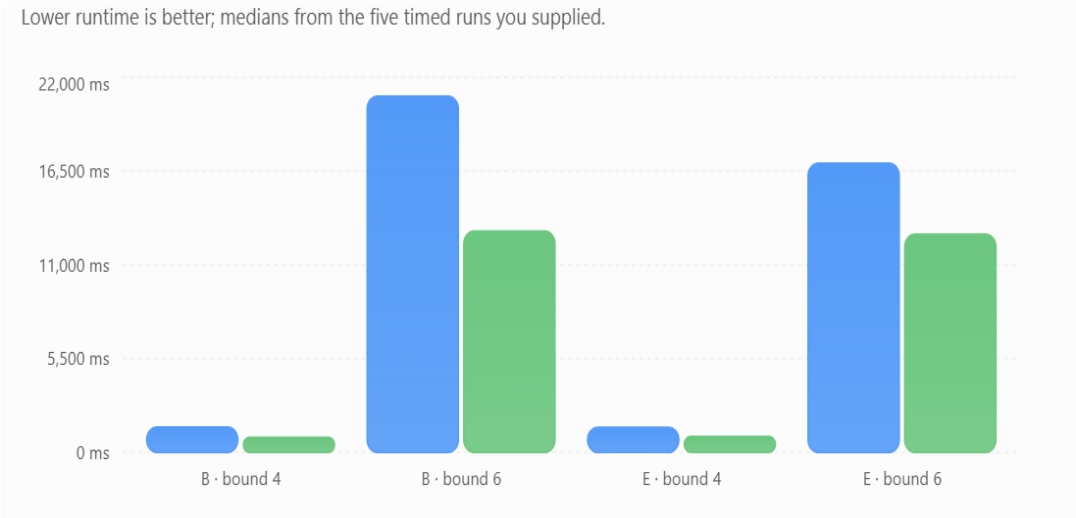


Figure 18. Q7 median runtime for B and E at bounds 4 and 6 (your supplied chart; lower is better).

7. Test concurrent requests

7.1 Building a small load test

The intended test compares two database engines, two workloads (Q4 and Q7), four client counts (1, 4, 8, and 16), and three repetitions, giving 48 combinations.

The primary measures are throughput, median latency, p95 latency, completed requests, errors, timeouts, and worker failures.

Python multiprocessing with one persistent connection per process and a Barrier; Q4 depth 10 and Q7 length-only bound 6 cycle through sets A–B–C with 1, 4, 8 and 16 clients; each client warms up once, runs for a 15 s deadline, finishes started cycles, keeps the 60 s timeout, and each combination is repeated three times. Throughput is correct completions divided by elapsed time.

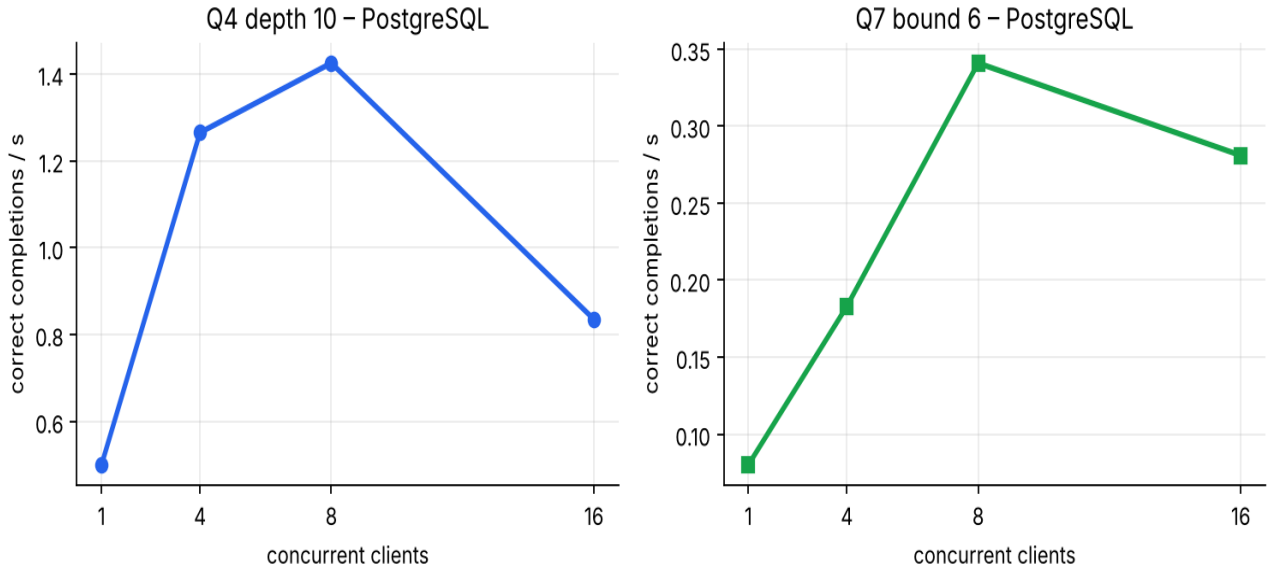


Figure 19. PostgreSQL throughput against clients (approximate values read from the supplied plots).

Neo4j: Q4 failed at 1 and 4 clients and has no result at 8 and 16; Q7 measurements are unavailable.

Q4 Throughput by Client Count

Requests per second



Q4 Concurrency Results

Clients	Throughput (requests/s)	Median latency (ms)	p95 latency (ms)
1	0.504	1,950.497	2,095.301
4	1.265	3,143.094	3,424.275
8	1.424	5,435.478	5,889.984
16	0.835	11,016.764	20,133.967

Q4 p95 Latency by Client Count

p95 latency (ms)

1 client	2,095.301
4 clients	3,424.275
8 clients	5,889.984
16 clients	20,133.967

7.2 Explanation of scaling

Throughput peaked at eight clients in the recorded values and then decreased at 16 clients. Tail latency rose sharply at 16 clients.



Figure 20. PostgreSQL p95 latency against clients (approximate values read from the supplied plots).

PostgreSQL throughput on Q4 rises from about 0.5 to 1.4 completions/s at 8 clients and falls to about 0.8 at 16, while p95 latency grows from about 2 s to 20 s. For Q7, throughput peaks near 0.34/s at 8 clients and falls at 16 while p95 rises from about 19 s to 76 s. Past 8 clients the 4-core machine is saturated, so extra clients queue: throughput (work per second) stops improving while each user's response time (latency) gets worse.

Q7 throughput peaked at eight clients and decreased at 16 clients. The p95 latency at 16 clients reached approximately 75.8 seconds.

8. Making and defend decisions

8.1 Prioritizing investigation

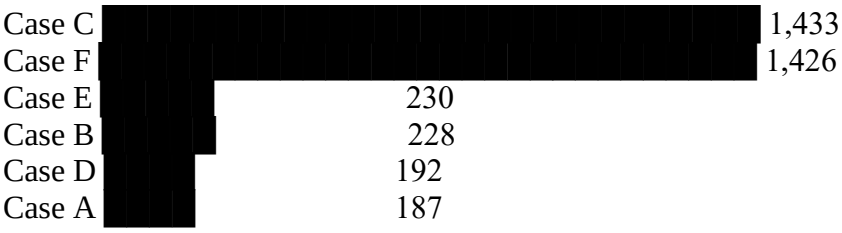
Investigation Workflow

Review account
|
v
Ownership and device-network traversal
|
v
Identify reachable accounts
|
v
Check shared-device interval overlap
|
v
Examine transfer chains or cycles
|
v
Validate timestamps, IDs, amounts and roles
|
v
Prioritize for human review

Investigators should consider account ownership, transaction context, device history, time overlap, and additional independent evidence before taking action.

The recorded depth-10 results show the largest reachable networks for cases C and F.

Prioritization by Recorded Depth-10 Network Size



Cases C and F should receive careful review of their network structure and supporting evidence. Cases A and C also have recorded temporal shared-device evidence.

This is an investigative prioritization, not a declaration of fraud. Shared devices and large networks can have legitimate explanations. Analysts should verify ownership, event times,

device-use intervals, transfer details, and other independent evidence before reaching conclusions.

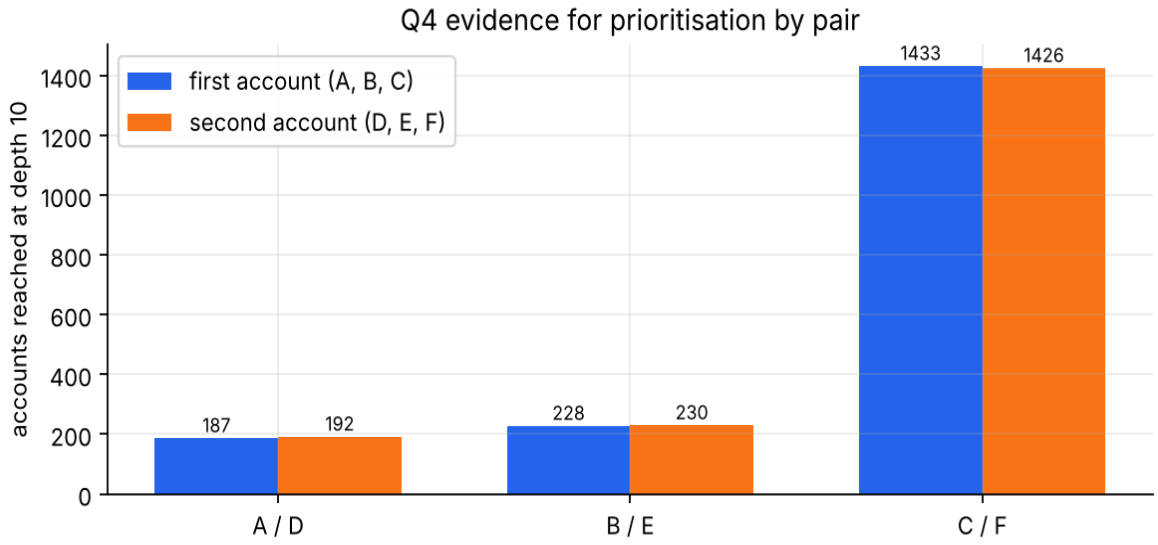


Figure 21. Accounts reached at depth 10 for each pair.

Pair	Q4 depth 10
A / D	187 vs 192
B / E	228 vs 230
C / F	1,433 vs 1,426

Q4 alone gives almost no separation inside each pair, so priority should rest on Q3 and Q5–Q7 evidence. One pair should be discussed in detail with IDs, amounts, times and device evidence, with one chain/cycle diagram and one ownership-device diagram reused from Section 5.

8.2.0 Recommending a model

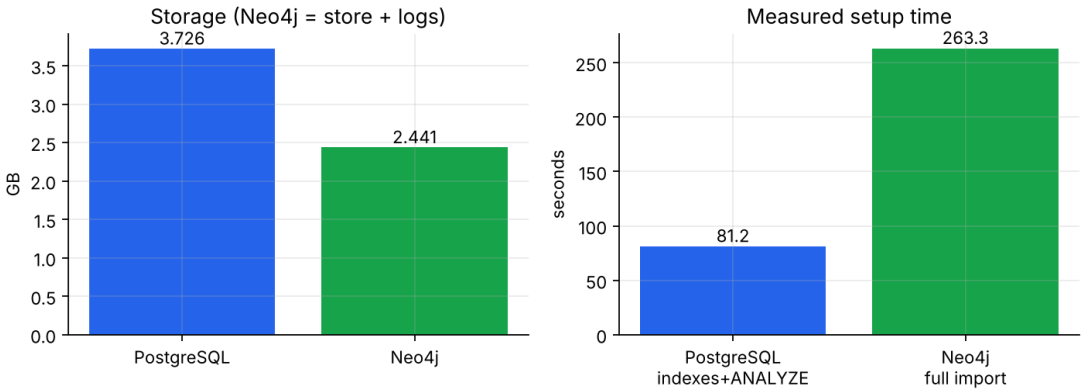


Figure 22. Measured storage and setup time, PostgreSQL vs Neo4j.

Provisional recommendation, based only on the evidence collected so far.

Reporting (Q1, Q2): PostgreSQL. These are indexed range scans and grouped aggregates, and the sender/receiver-time indexes support them directly. Traversal (Q4, Q7): the graph model expresses variable-length paths more simply than a recursive CTE, but the supplied evidence does not show a performance win: Neo4j Q4 failed in the load test and no Neo4j Q7 measurements exist, while PostgreSQL ran both. Concurrency: PostgreSQL saturated at about 8 clients on this four-core laptop, so neither engine can be called scalable from this experiment.

Load time and storage: PostgreSQL needed 3.726 GB and about 81 s of index/ANALYZE time after loading; Neo4j needed 2.441 GB (store plus logs) and 263 s to import. Neo4j is smaller, but PostgreSQL's transfer indexes alone (2.43 GB) exceed the table, the price of fast time-filtered lookups.

Model versus implementation: path expressiveness belongs to the graph model, while runtime depends on the algorithm. The Q7 timing chart in 6.3 shows a clear gap between two SQL versions at bound 6, which suggests that part of a slow result is SQL implementation, not the relational model (confirm once the series are labelled).

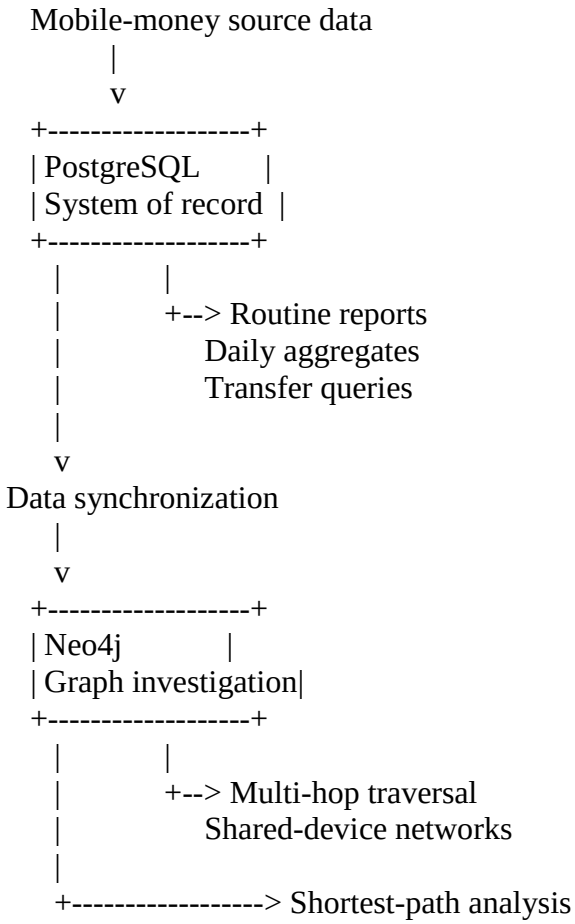
Evidence against my preferred choice: the graph model's readable path queries and the smaller store favour Neo4j for investigation work.

Freshness: if operational data is copied into a graph, I propose a target of at most 15 minutes behind the source. Stale data could miss a new transfer or shared device, understating a chain or connection and delaying a review.

8.2.1 Database Architecture Recommendation

A hybrid architecture is recommended based on the recorded measurements.

Recommended Architecture



PostgreSQL should be used for structured transaction records, routine reporting, and aggregation. Neo4j should complement it for multi-hop network investigations and shortest-path analysis.

A proposed freshness target is no more than 60 seconds between PostgreSQL updates and availability of the corresponding graph data. This target must be tested and monitored. Storage consumption, full data-loading time, and complete concurrent performance comparisons were not available in the recorded evidence.

8.3 Individual Reflections and Contributions

Victor Dusengimana; with Student ID 101372

The recorded tests show that database performance depends on the workload. PostgreSQL performed well in several transfer oriented queries, while Neo4j was faster in the measured network traversal and shortest path tests. A further check that could change this conclusion is a controlled repeat of the tests with identical data, equivalent query rules, successful repetitions, and complete timing logs. My contribution focuses on interpreting the measured performance and ensuring that incomplete results are not presented as successful tests.

Umwiza Odile; with Student ID 101365

The device investigation demonstrates that shared device evidence can reveal connections between accounts, but it cannot independently establish wrongdoing. A device may be used by family members, employees, businesses, or legitimate users. A further check that could change an account's priority is to verify the original device identifiers, device types, use intervals, and account ownership. My contribution focuses on interpreting device evidence carefully and distinguishing an investigative lead from a confirmed conclusion.

Igirukwayo Ildephonse; with Student ID 101404

The project supports a hybrid database approach. PostgreSQL provides a useful structure for transaction records and aggregation, while Neo4j naturally represents connected accounts and devices. The recorded performance results support using each database for the workloads where it performed well. A further check that could change the recommendation is a complete concurrent benchmark with successful measurements from both systems, together with storage, loading, synchronization, and recovery measurements. My contribution focuses on connecting the query results with the architecture recommendation.

Mukiza Alain Christian; with Student ID 101370

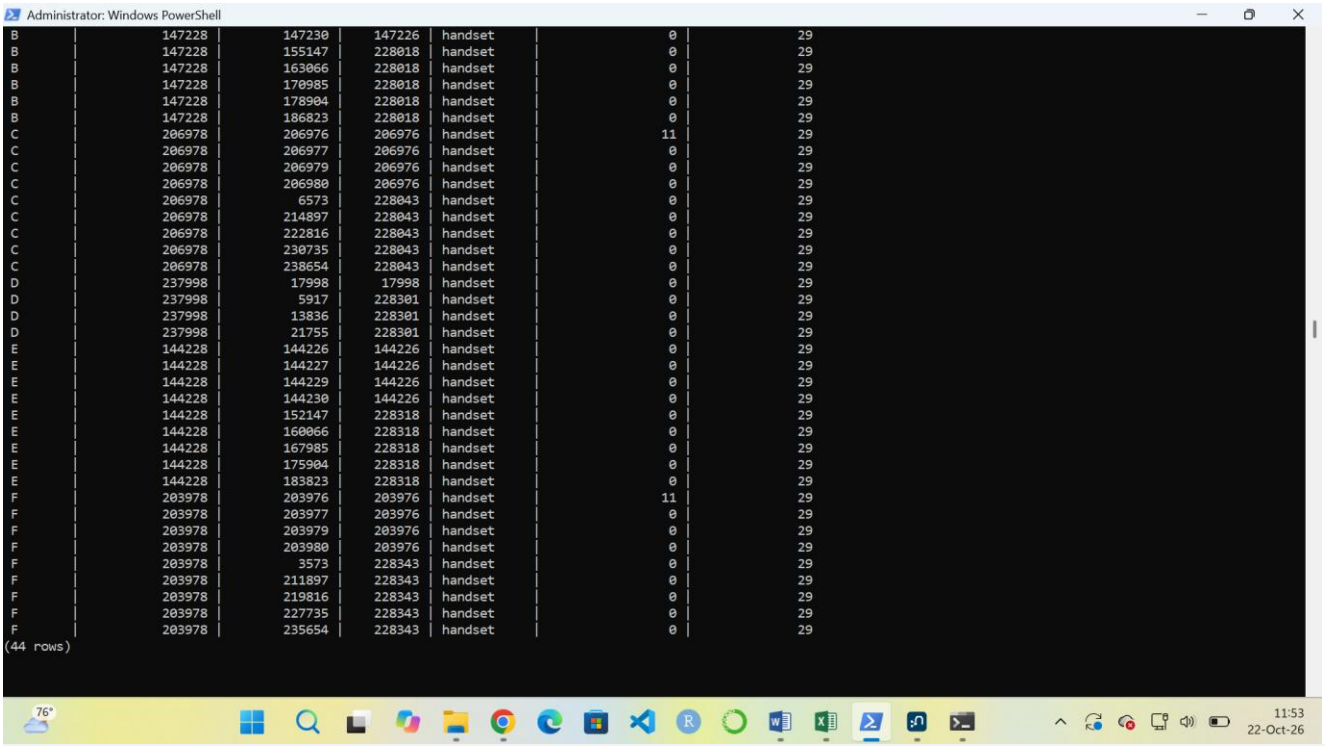
The depth-10 results show that cases C and F have much larger reachable networks than the other review cases. Network size can help prioritize investigations, but it is not proof of fraud. A further check that could change the prioritization is to inspect the full account and device paths, verify relationship direction and temporal overlap, and compare the findings with independent records. My contribution focuses on the responsible interpretation of graph evidence and the need for traceable validation.

Overall conclusion

The project captured 20,993,500 records in 168.8 s with every count and the transfer audit (20,003,300 transfers, 231,482,327,870 RWF) matching. PostgreSQL loaded them into a constrained five-table schema of 3.726 GB, and Neo4j imported 704,600 nodes and 20,536,900 relationships in 263 s into 2.441 GB; roles, ownership and transfer-ID uniqueness agree across both. Q4 showed that reachable accounts grow with depth, most strongly for sets C and F, and deeper links add weaker associations. In the concurrency tests PostgreSQL saturated at about 8 clients, and the Neo4j load-test results are incomplete, so this experiment does not establish a graph performance advantage. PostgreSQL is the safer choice for reporting; Neo4j is attractive for readable path investigations, but its recommendation here is provisional. Network links are leads for review, not proof of fraud.

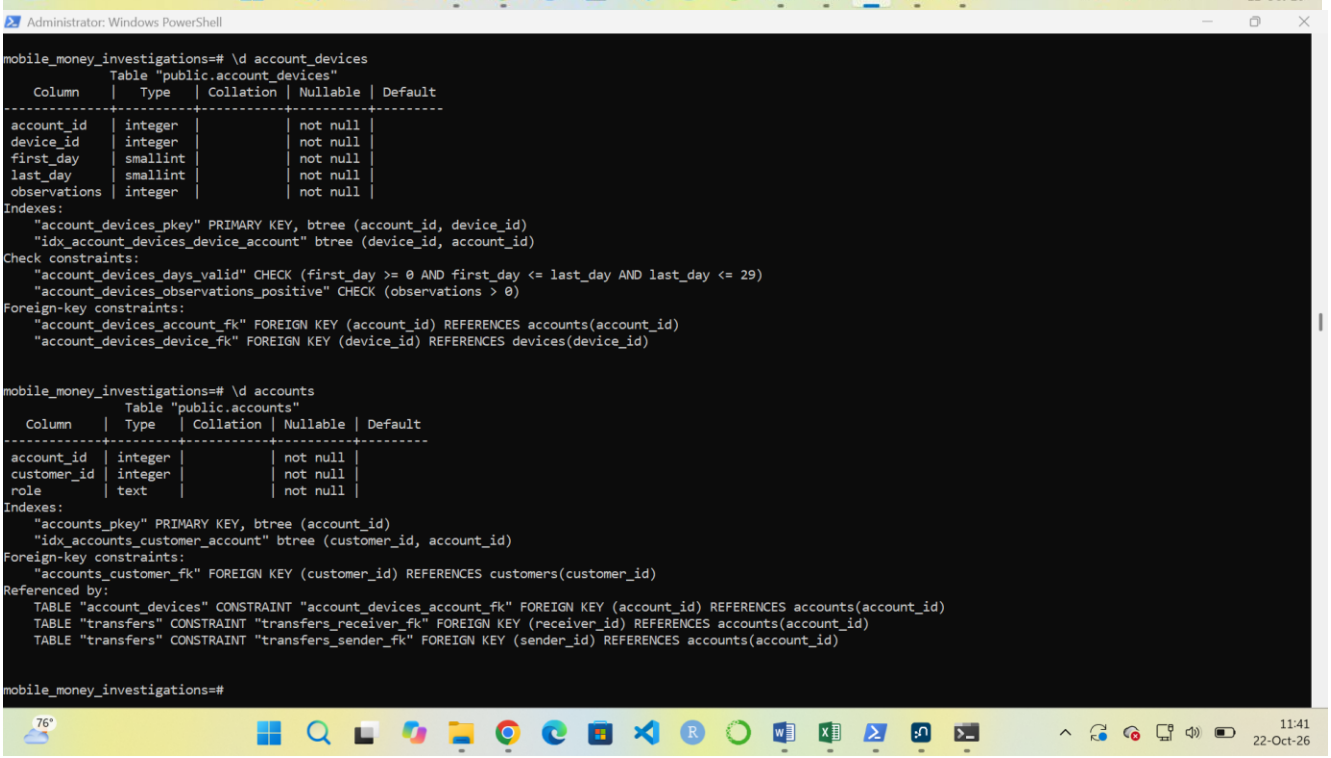
APPENDIX:

Group 3_ Mobile Money Investigations with PostgreSQL and Neo4j_project_report



```
Administrator: Windows PowerShell

 B       |          147228 |        147230 |    147226 | handset     |                0 |             29
 B       |          147228 |        155147 |    228018 | handset     |                0 |             29
 B       |          147228 |        163066 |    228018 | handset     |                0 |             29
 B       |          147228 |        170985 |    228018 | handset     |                0 |             29
 B       |          147228 |        178904 |    228018 | handset     |                0 |             29
 B       |          147228 |        186823 |    228018 | handset     |                0 |             29
 C       |          206978 |        206976 |    206976 | handset     |               11 |             29
 C       |          206978 |        206977 |    206976 | handset     |                0 |             29
 C       |          206978 |        206979 |    206976 | handset     |                0 |             29
 C       |          206978 |        206980 |    206976 | handset     |                0 |             29
 C       |          206978 |          6573 |    228043 | handset     |                0 |             29
 C       |          206978 |        214897 |    228043 | handset     |                0 |             29
 C       |          206978 |        222816 |    228043 | handset     |                0 |             29
 C       |          206978 |        230735 |    228043 | handset     |                0 |             29
 C       |          206978 |        238654 |    228043 | handset     |                0 |             29
 D       |          237998 |         17998 |     17998 | handset     |                0 |             29
 D       |          237998 |          5917 |    228301 | handset     |                0 |             29
 D       |          237998 |         13836 |    228301 | handset     |                0 |             29
 D       |          237998 |         21755 |    228301 | handset     |                0 |             29
 E       |          144228 |        144226 |    144226 | handset     |                0 |             29
 E       |          144228 |        144227 |    144226 | handset     |                0 |             29
 E       |          144228 |        144229 |    144226 | handset     |                0 |             29
 E       |          144228 |        144230 |    144226 | handset     |                0 |             29
 E       |          144228 |        152147 |    228318 | handset     |                0 |             29
 E       |          144228 |        160066 |    228318 | handset     |                0 |             29
 E       |          144228 |        167985 |    228318 | handset     |                0 |             29
 E       |          144228 |        175904 |    228318 | handset     |                0 |             29
 E       |          144228 |        183823 |    228318 | handset     |                0 |             29
 F       |          203978 |        203976 |    203976 | handset     |               11 |             29
 F       |          203978 |        203977 |    203976 | handset     |                0 |             29
 F       |          203978 |        203979 |    203976 | handset     |                0 |             29
 F       |          203978 |        203980 |    203976 | handset     |                0 |             29
 F       |          203978 |          3573 |    228343 | handset     |                0 |             29
 F       |          203978 |        211897 |    228343 | handset     |                0 |             29
 F       |          203978 |        219816 |    228343 | handset     |                0 |             29
 F       |          203978 |        227735 |    228343 | handset     |                0 |             29
 F       |          203978 |        235654 |    228343 | handset     |                0 |             29
(44 rows)
```



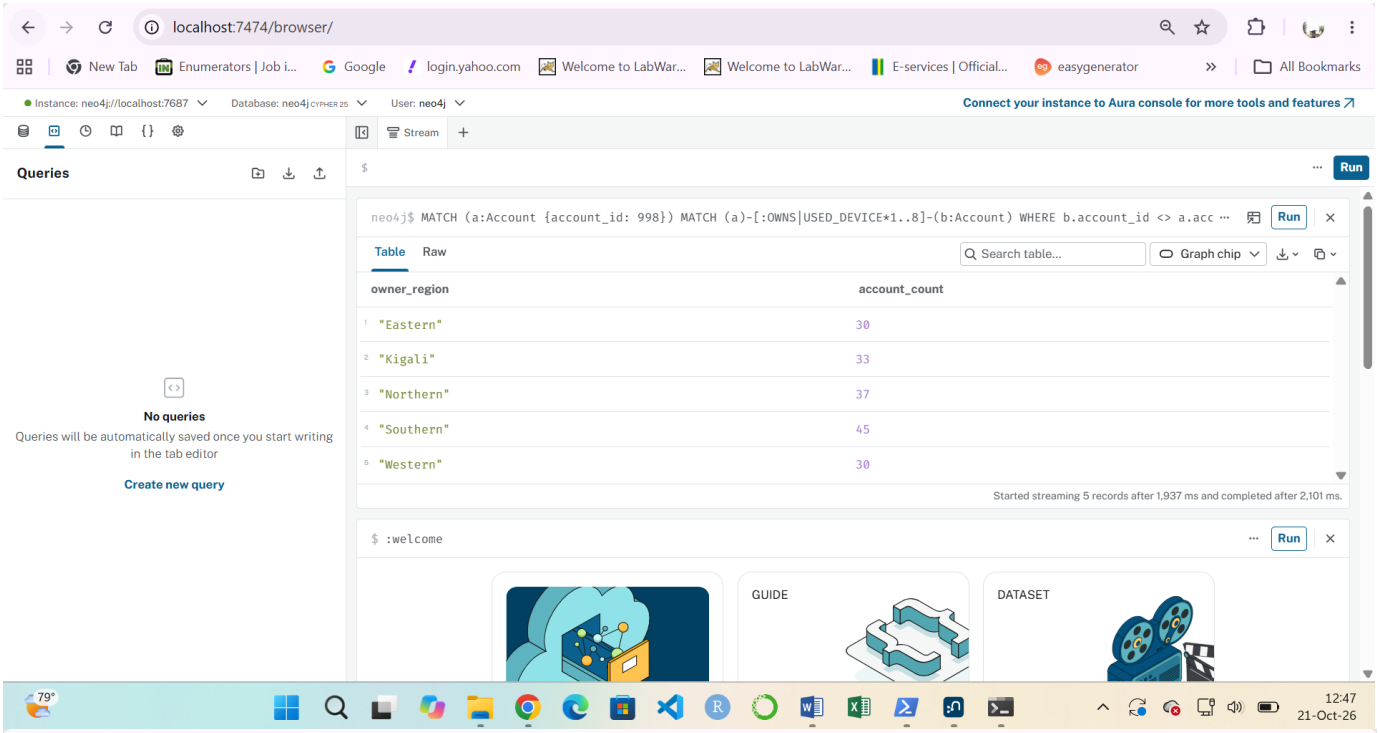
```
Administrator: Windows PowerShell

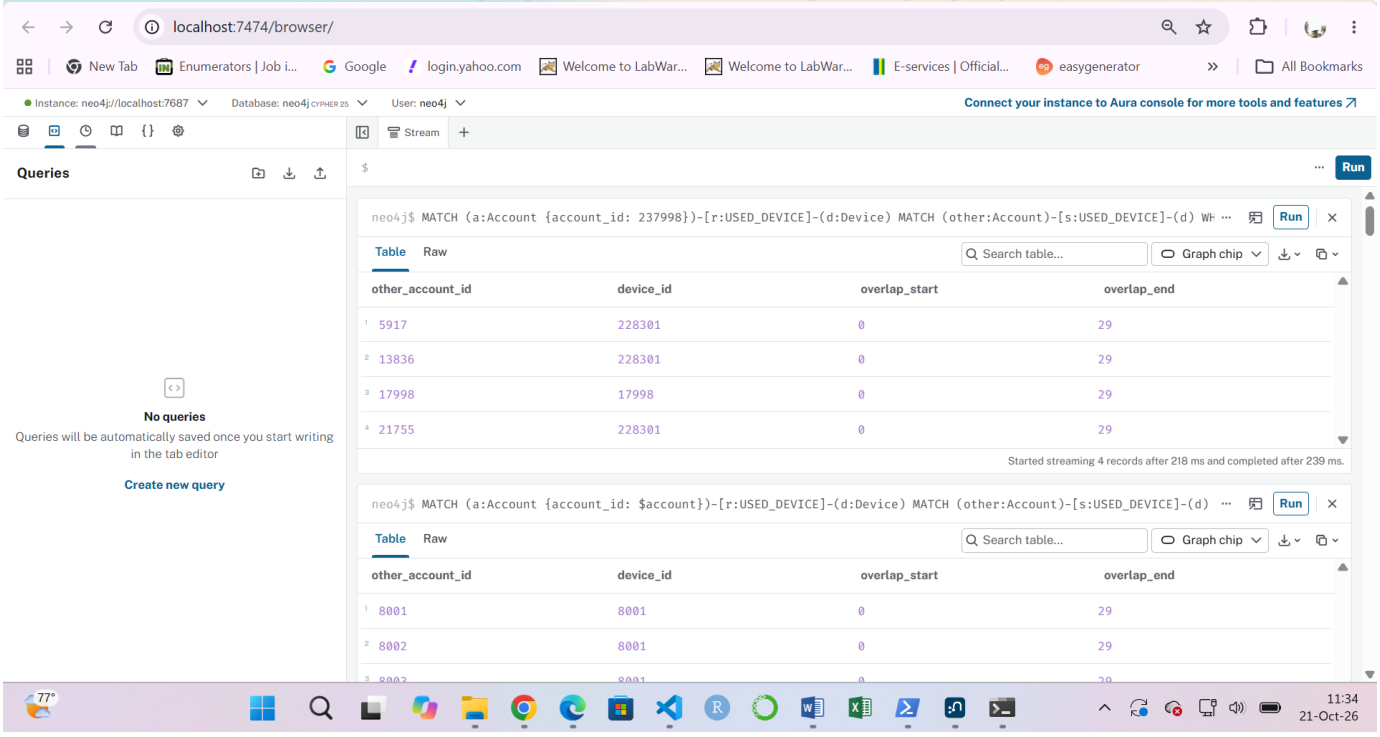
mobile_money_investigations=# \d account_devices
                Table "public.account_devices"
    Column     |   Type   | Collation | Nullable | Default
---------------+----------+-----------+----------+---------
 account_id    | integer  |           | not null |
 device_id     | integer  |           | not null |
 first_day     | smallint |           | not null |
 last_day      | smallint |           | not null |
 observations  | integer  |           | not null |
Indexes:
    "account_devices_pkey" PRIMARY KEY, btree (account_id, device_id)
    "idx_account_devices_device_account" btree (device_id, account_id)
Check constraints:
    "account_devices_days_valid" CHECK (first_day >= 0 AND first_day <= last_day AND last_day <= 29)
    "account_devices_observations_positive" CHECK (observations > 0)
Foreign-key constraints:
    "account_devices_account_fk" FOREIGN KEY (account_id) REFERENCES accounts(account_id)
    "account_devices_device_fk" FOREIGN KEY (device_id) REFERENCES devices(device_id)


mobile_money_investigations=# \d accounts
                  Table "public.accounts"
   Column    |  Type   | Collation | Nullable | Default
-------------+---------+-----------+----------+---------
 account_id  | integer |           | not null |
 customer_id | integer |           | not null |
 role        | text    |           | not null |
Indexes:
    "accounts_pkey" PRIMARY KEY, btree (account_id)
    "idx_accounts_customer_account" btree (customer_id, account_id)
Foreign-key constraints:
    "accounts_customer_fk" FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
Referenced by:
    TABLE "account_devices" CONSTRAINT "account_devices_account_fk" FOREIGN KEY (account_id) REFERENCES accounts(account_id)
    TABLE "transfers" CONSTRAINT "transfers_receiver_fk" FOREIGN KEY (receiver_id) REFERENCES accounts(account_id)
    TABLE "transfers" CONSTRAINT "transfers_sender_fk" FOREIGN KEY (sender_id) REFERENCES accounts(account_id)


mobile_money_investigations=#
```

Group 3_ Mobile Money Investigations with PostgreSQL and Neo4j_project_report

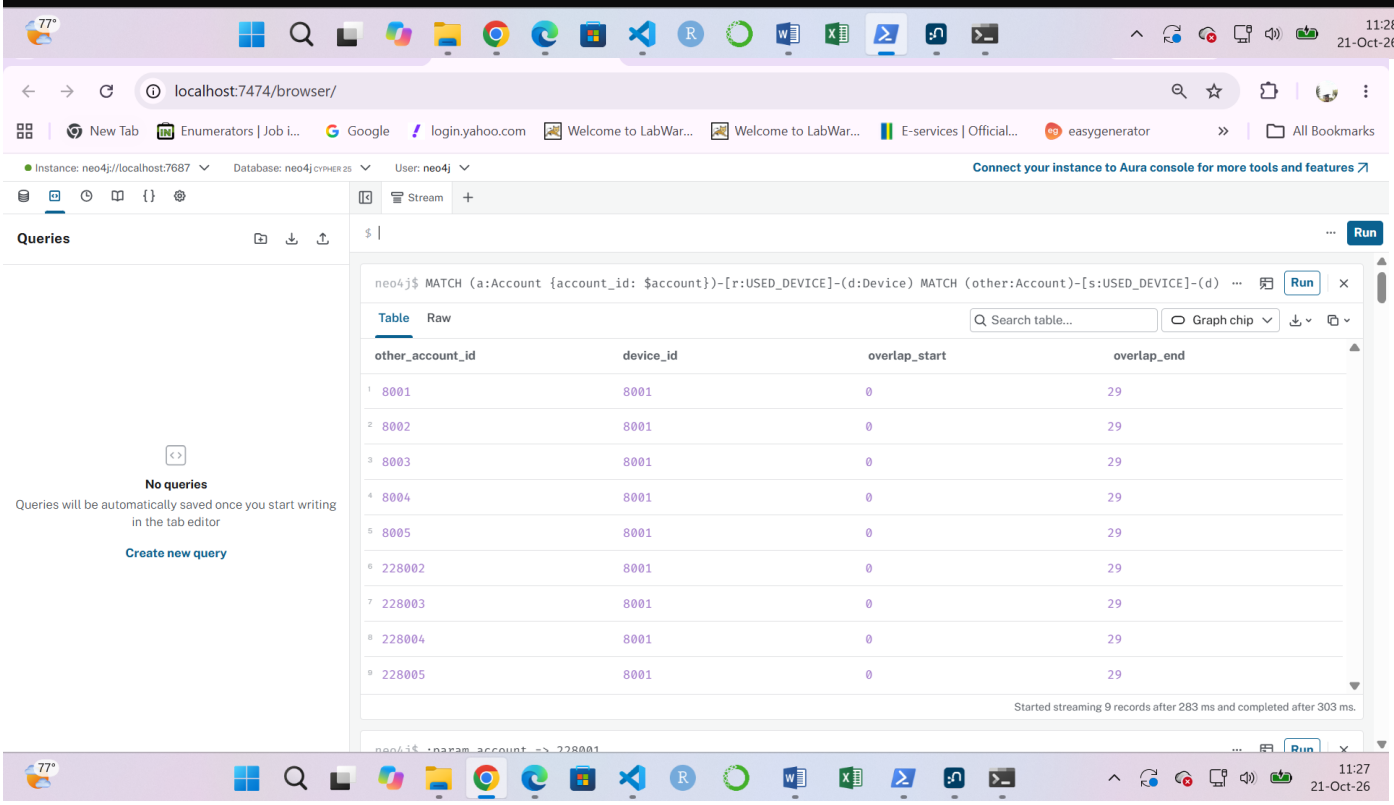




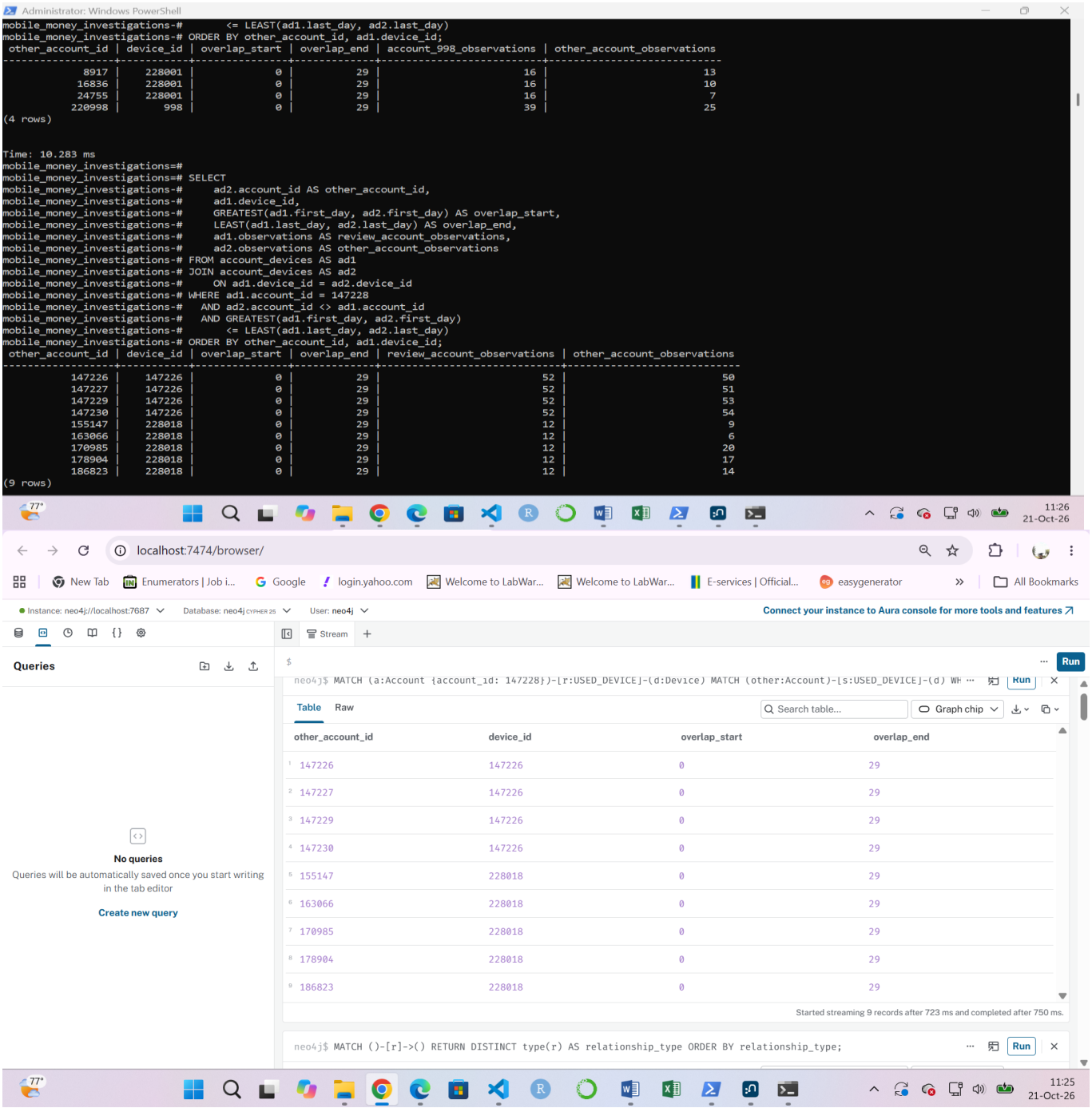
Group 3_ Mobile Money Investigations with PostgreSQL and Neo4j_project_report

```
mobile_money_investigations-# ORDER BY other_account_id, ad1.device_id;
 other_account_id | device_id | overlap_start | overlap_end | review_account_observations | other_account_observations
------------------+-----------+---------------+-------------+-----------------------------+----------------------------
             8001 |      8001 |             0 |          29 |                          41 |                         55
             8002 |      8001 |             0 |          29 |                          41 |                         56
             8003 |      8001 |             0 |          29 |                          41 |                         57
             8004 |      8001 |             0 |          29 |                          41 |                         58
             8005 |      8001 |             0 |          29 |                          41 |                         59
           228002 |      8001 |             0 |          29 |                          41 |                         42
           228003 |      8001 |             0 |          29 |                          41 |                         43
           228004 |      8001 |             0 |          29 |                          41 |                         44
           228005 |      8001 |             0 |          29 |                          41 |                         45
(9 rows)


Time: 4.865 ms
mobile_money_investigations=#
```



Group 3_ Mobile Money Investigations with PostgreSQL and Neo4j_project_report

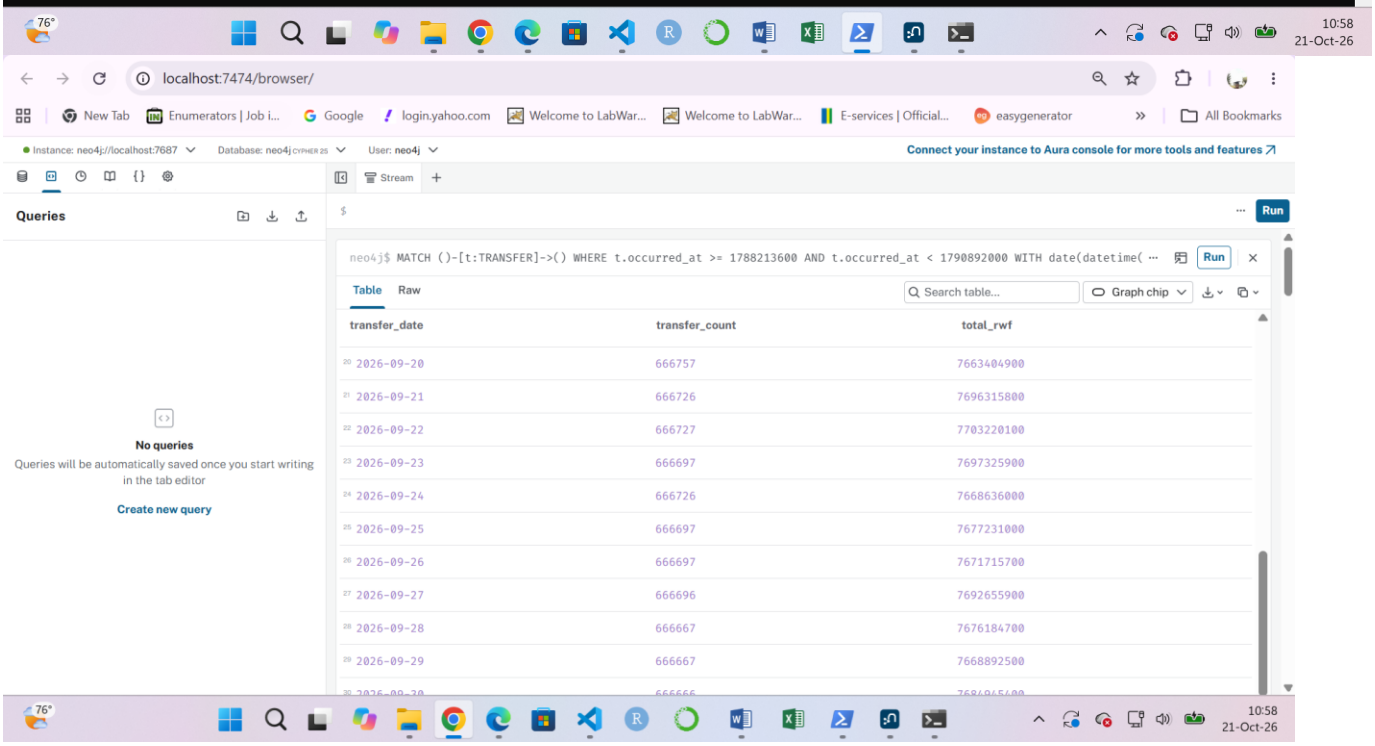


```
Administrator: Windows PowerShell
mobile_money_investigations-#       <= LEAST(ad1.last_day, ad2.last_day)
mobile_money_investigations-# ORDER BY other_account_id, ad1.device_id;
 other_account_id | device_id | overlap_start | overlap_end | account_998_observations | other_account_observations
------------------+-----------+---------------+-------------+--------------------------+----------------------------
             8917 |    228001 |             0 |          29 |                       16 |                         13
            16836 |    228001 |             0 |          29 |                       16 |                         10
            24755 |    228001 |             0 |          29 |                       16 |                          7
           220998 |       998 |             0 |          29 |                       39 |                         25
(4 rows)

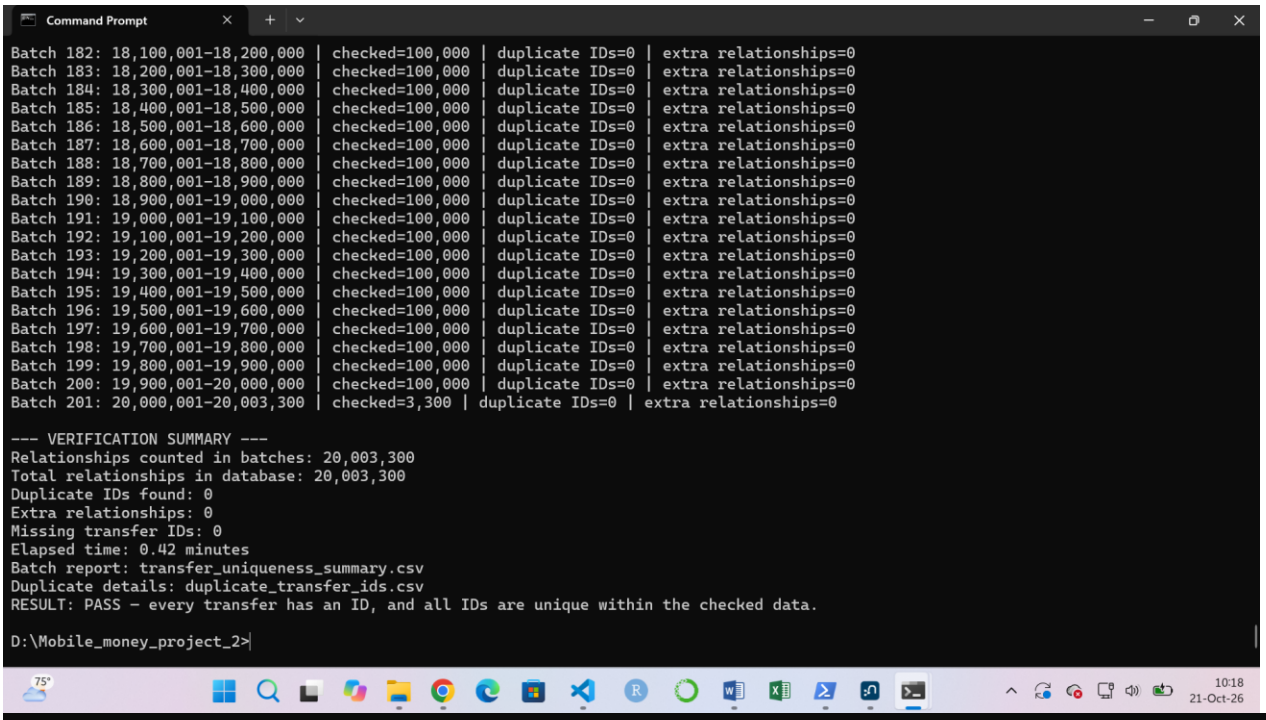

Time: 10.283 ms
mobile_money_investigations=#
mobile_money_investigations=# SELECT
mobile_money_investigations-#     ad2.account_id AS other_account_id,
mobile_money_investigations-#     ad1.device_id,
mobile_money_investigations-#     GREATEST(ad1.first_day, ad2.first_day) AS overlap_start,
mobile_money_investigations-#     LEAST(ad1.last_day, ad2.last_day) AS overlap_end,
mobile_money_investigations-#     ad1.observations AS review_account_observations,
mobile_money_investigations-#     ad2.observations AS other_account_observations
mobile_money_investigations-# FROM account_devices AS ad1
mobile_money_investigations-# JOIN account_devices AS ad2
mobile_money_investigations-#     ON ad1.device_id = ad2.device_id
mobile_money_investigations-# WHERE ad1.account_id = 147228
mobile_money_investigations-#   AND ad2.account_id <> ad1.account_id
mobile_money_investigations-#   AND GREATEST(ad1.first_day, ad2.first_day)
mobile_money_investigations-#       <= LEAST(ad1.last_day, ad2.last_day)
mobile_money_investigations-# ORDER BY other_account_id, ad1.device_id;
 other_account_id | device_id | overlap_start | overlap_end | review_account_observations | other_account_observations
------------------+-----------+---------------+-------------+-----------------------------+----------------------------
           147226 |    147226 |             0 |          29 |                          52 |                         50
           147227 |    147226 |             0 |          29 |                          52 |                         51
           147229 |    147226 |             0 |          29 |                          52 |                         53
           147230 |    147226 |             0 |          29 |                          52 |                         54
           155147 |    228018 |             0 |          29 |                          12 |                          9
           163066 |    228018 |             0 |          29 |                          12 |                          6
           170985 |    228018 |             0 |          29 |                          12 |                         20
           178904 |    228018 |             0 |          29 |                          12 |                         17
           186823 |    228018 |             0 |          29 |                          12 |                         14
(9 rows)
```

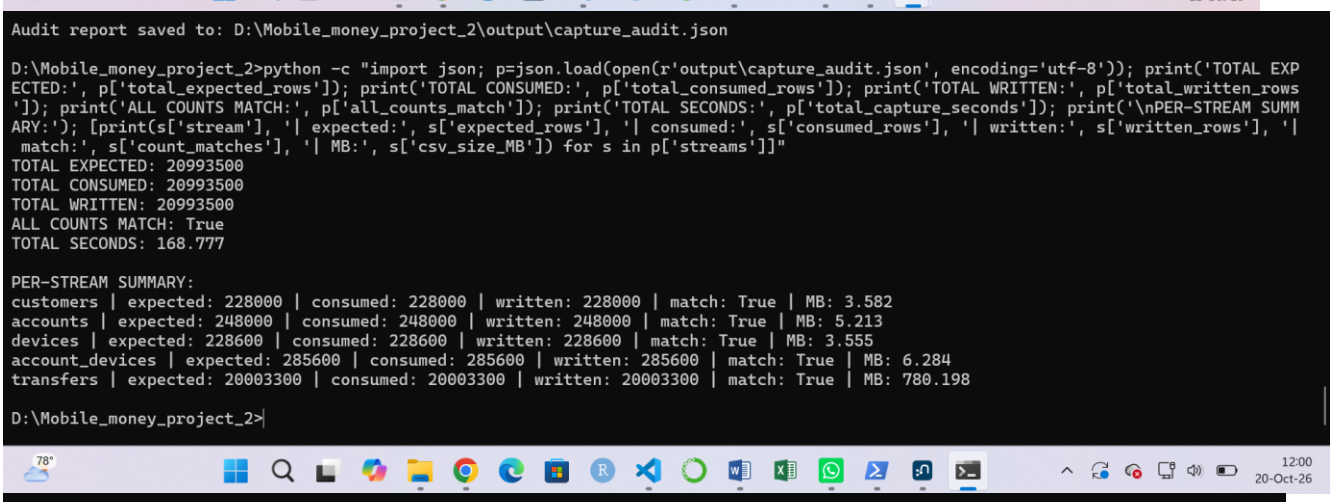
Group 3_ Mobile Money Investigations with PostgreSQL and Neo4j_project_report

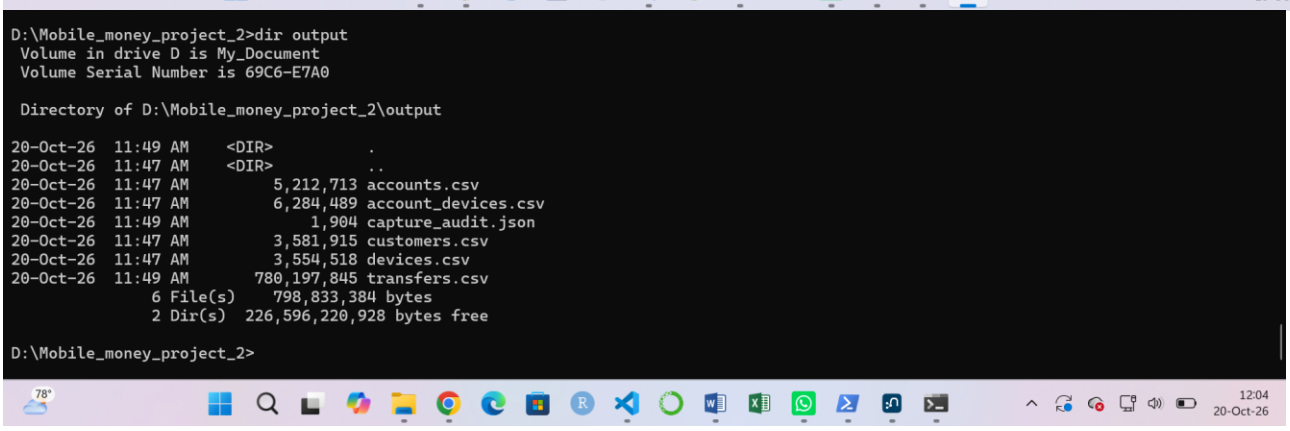
```
mobile_money_investigations-# GROUP BY transfer_date
mobile_money_investigations-# ORDER BY transfer_date;
 transfer_date | transfer_count | total_rwf
---------------+----------------+------------
 2026-09-01    |         666667 | 7683326000
 2026-09-02    |         666667 | 7678846900
 2026-09-03    |         666666 | 7665877900
 2026-09-04    |         666667 | 7665199000
 2026-09-05    |         666667 | 7679091900
 2026-09-06    |         666816 | 7741994180
 2026-09-07    |         666817 | 7726576180
 2026-09-08    |         666907 | 7777047990
 2026-09-09    |         666906 | 7783400210
 2026-09-10    |         666997 | 7807170170
 2026-09-11    |         666847 | 7746125100
 2026-09-12    |         666936 | 7783593400
 2026-09-13    |         666937 | 7800995510
 2026-09-14    |         666997 | 7856644775
 2026-09-15    |         666996 | 7859428355
 2026-09-16    |         666787 | 7683087200
 2026-09-17    |         666757 | 7699710100
 2026-09-18    |         666786 | 7677843700
 2026-09-19    |         666757 | 7665841400
 2026-09-20    |         666757 | 7663404900
 2026-09-21    |         666726 | 7696315800
 2026-09-22    |         666727 | 7703220100
 2026-09-23    |         666697 | 7697325900
 2026-09-24    |         666726 | 7668636000
 2026-09-25    |         666697 | 7677231000
 2026-09-26    |         666697 | 7671715700
 2026-09-27    |         666696 | 7692655900
 2026-09-28    |         666667 | 7676184700
 2026-09-29    |         666667 | 7668892500
 2026-09-30    |         666666 | 7684945400
(30 rows)
```



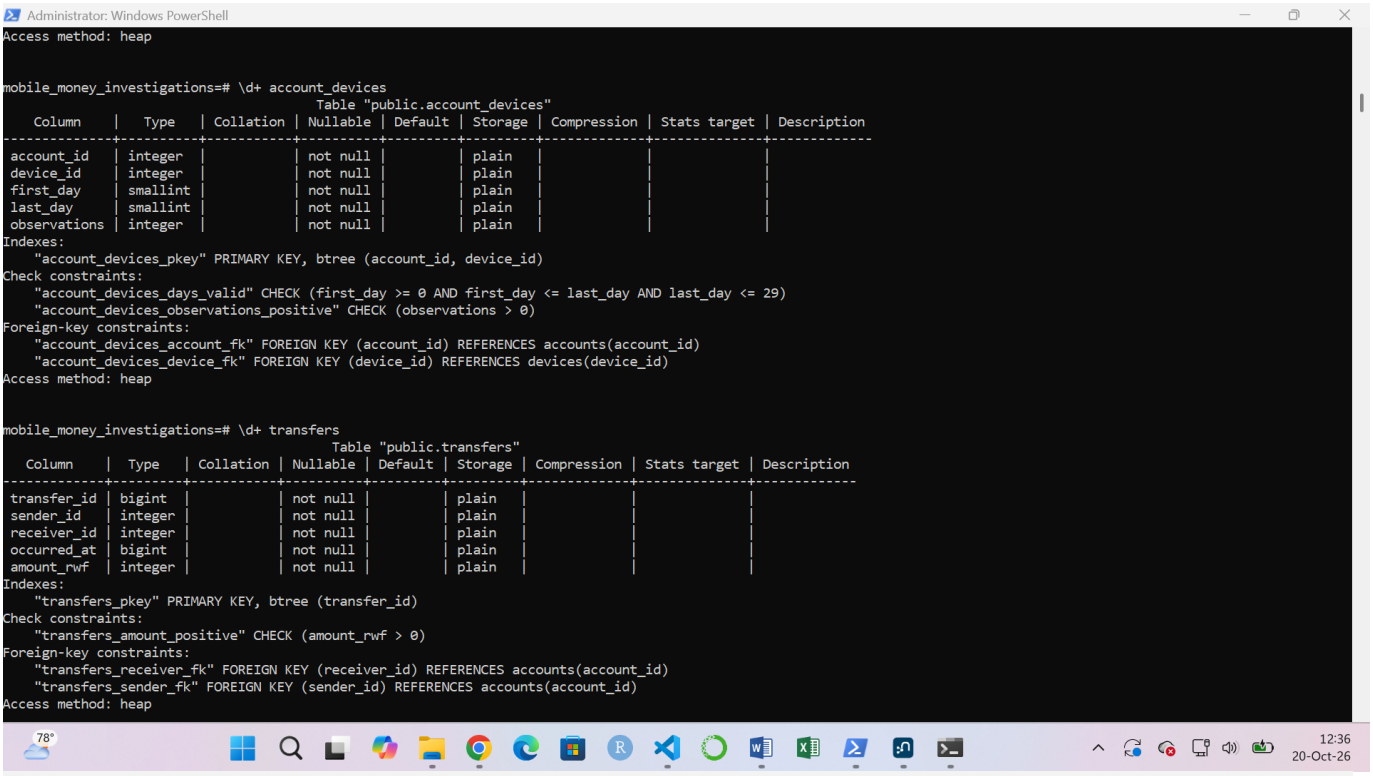
Group 3_ Mobile Money Investigations with PostgreSQL and Neo4j_project_report

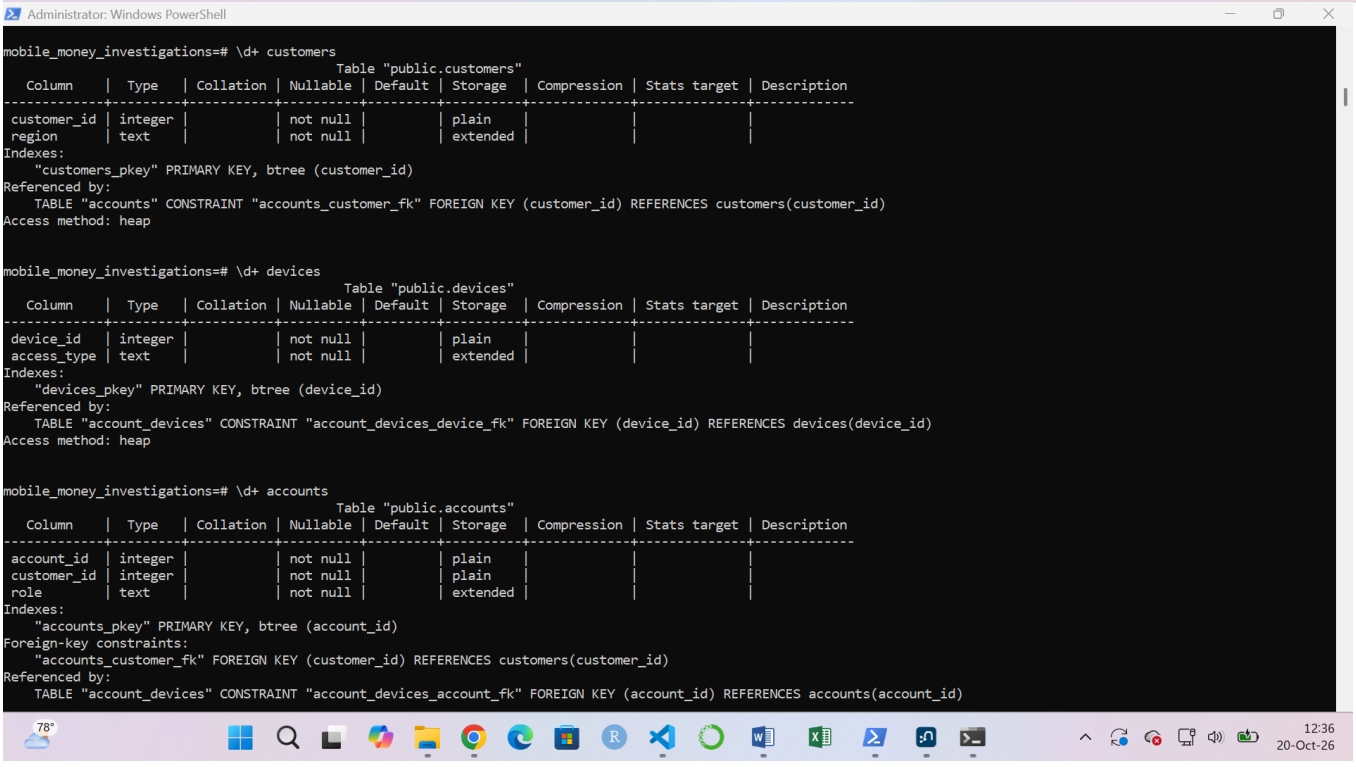






Group 3_ Mobile Money Investigations with PostgreSQL and Neo4j_project_report

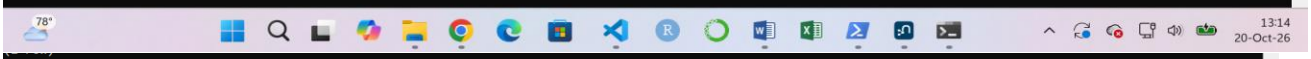




Group 3_ Mobile Money Investigations with PostgreSQL and Neo4j_project_report

```
mobile_money_investigations=#
mobile_money_investigations=# SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM customers
mobile_money_investigations-# UNION ALL
mobile_money_investigations-# SELECT 'devices', COUNT(*) FROM devices
mobile_money_investigations-# UNION ALL
mobile_money_investigations-# SELECT 'accounts', COUNT(*) FROM accounts
mobile_money_investigations-# UNION ALL
mobile_money_investigations-# SELECT 'account_devices', COUNT(*) FROM account_devices
mobile_money_investigations-# UNION ALL
mobile_money_investigations-# SELECT 'transfers', COUNT(*) FROM transfers;
   table_name    | row_count
-----------------+-----------
 customers       |    228000
 devices         |    228600
 accounts        |    248000
 account_devices |    285600
 transfers       |  20003300
(5 rows)

mobile_money_investigations=#
```

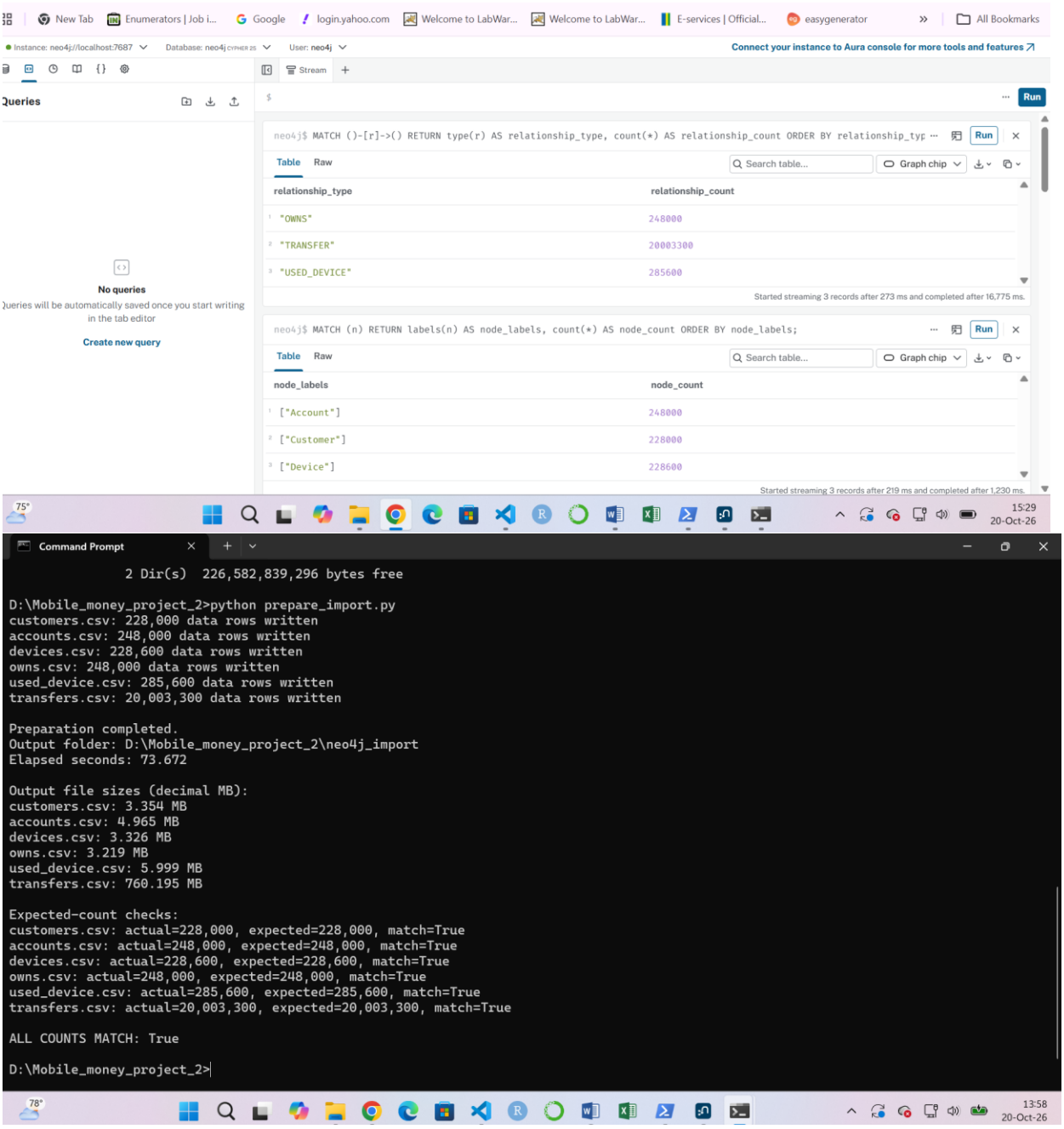


```
Time: 153.327 ms
mobile_money_investigations=#
mobile_money_investigations=# SELECT
mobile_money_investigations-#     conrelid::regclass AS table_name,
mobile_money_investigations-#     conname AS constraint_name,
mobile_money_investigations-#     CASE contype
mobile_money_investigations-#         WHEN 'p' THEN 'PRIMARY KEY'
mobile_money_investigations-#         WHEN 'f' THEN 'FOREIGN KEY'
mobile_money_investigations-#         WHEN 'c' THEN 'CHECK'
mobile_money_investigations-#         WHEN 'u' THEN 'UNIQUE'
mobile_money_investigations-#         ELSE contype::text
mobile_money_investigations-#     END AS constraint_type,
mobile_money_investigations-#     convalidated AS validated
mobile_money_investigations-# FROM pg_constraint
mobile_money_investigations-# WHERE connamespace = 'public'::regnamespace
mobile_money_investigations-# ORDER BY table_name, constraint_name;
   table_name    |           constraint_name            | constraint_type | validated
-----------------+--------------------------------------+-----------------+-----------
 customers       | customers_pkey                       | PRIMARY KEY     | t
 devices         | devices_pkey                         | PRIMARY KEY     | t
 accounts        | accounts_customer_fk                 | FOREIGN KEY     | t
 accounts        | accounts_pkey                        | PRIMARY KEY     | t
 account_devices | account_devices_account_fk           | FOREIGN KEY     | t
 account_devices | account_devices_days_valid           | CHECK           | t
 account_devices | account_devices_device_fk            | FOREIGN KEY     | t
 account_devices | account_devices_observations_positive | CHECK           | t
 account_devices | account_devices_pkey                 | PRIMARY KEY     | t
 transfers       | transfers_amount_positive            | CHECK           | t
 transfers       | transfers_pkey                       | PRIMARY KEY     | t
 transfers       | transfers_receiver_fk                | FOREIGN KEY     | t
 transfers       | transfers_sender_fk                  | FOREIGN KEY     | t
(13 rows)

Time: 10.810 ms
mobile_money_investigations=#
```



Group 3_ Mobile Money Investigations with PostgreSQL and Neo4j_project_report



Group 3_ Mobile Money Investigations with PostgreSQL and Neo4j_project_report

